

СОДЕРЖАНИЕ

Введение.....	6
1 Обзор литературы	7
1.1 Исследование предметной области.....	7
1.2 Обзор аналогов	9
1.3 Обзор средств разработки	17
1.4 База данных.....	23
1.5 Концепция разрабатываемой игровой системы.....	25
1.6 Выводы.....	26
2 Системное проектирование.....	28
2.1 Обоснование выбора средств разработки.....	29
3 Функциональное проектирование	40
3.1 Описание структуры классов.....	40
7 Технико-экономическое обоснование разработки программного модуля адаптивного поведения персонажей 2D-игры в жанре ActionRPG	61
7.1 Характеристика программного средства, разрабатываемого по индивидуальному заказу	61
7.2 Расчет инвестиций в разработку программного средства	62
7.3 Расчет разработки и использования программного средства.....	65
7.4 Расчет показателей экономической эффективности разработки программного средства.....	66
7.5 Вывод об экономической целесообразности реализации проектного решения	66
Заключение	68
Список использованных источников	69
Приложение А	70
Приложение Б.....	71
Приложение В.....	72

ВВЕДЕНИЕ

Игровые программные системы с процедурной организацией пространства и повторяемым циклом прохождения предъявляют требования к поведению противников. При фиксированных шаблонах реакции игровой процесс становится предсказуемым, поскольку последовательность действий противника повторяется при одинаковых условиях и в одинаковой конфигурации игрового пространства. Для проектов жанра ActionRPG это снижает вариативность боевых сценариев уже на ранних этапах взаимодействия игрока с системой и сокращает различие между отдельными игровыми столкновениями.

В рамках проектирования игровых систем данного типа задача управления противниками обычно решается через заранее заданные сценарии поведения, основанные на простых проверках расстояния до цели и факта обнаружения игрока. Такой подход обеспечивает предсказуемость реализации, однако не учитывает изменение состояния противника, плотность окружения и последовательность действий игрока. Для сохранения различий между боевыми ситуациями требуется отдельный программный модуль выбора поведения по совокупности входных параметров, способный изменять реакцию противника в одинаковых игровых условиях при различной последовательности действий игрока.

Игровая система используется как среда функционирования разработанного алгоритма и обеспечивает проверку его работы в условиях игрового цикла, включающего перемещение между комнатами, боевые взаимодействия, повторные прохождения и накопление прогресса между попытками. Основная часть проектирования сосредоточена на построении собственной схемы выбора поведения противников без применения встроенных средств описания поведения игрового движка.

Целью дипломного проекта является разработка программного модуля адаптивного поведения противников и создание двумерной игровой среды в жанре ActionRPG для его функционирования, обеспечивающих изменение состояний поведения противников на основе параметров игровой ситуации и последних действий игрока.

В соответствии с поставленной целью определены следующие задачи:

- разработать игровую среду и базовые игровые механики для функционирования программного модуля;
- спроектировать конечный автомат состояний поведения противников;
- реализовать модуль адаптивного выбора поведения на основе оценки угрозы и памяти действий игрока.

Данный дипломный проект выполнен мной лично, проверен на заимствования, процент оригинальности составляет XX% (отчет о проверке на заимствования прилагается).

1 ОБЗОР ЛИТЕРАТУРЫ

1.1 Исследование предметной области

Жанр ActionRPG представляет собой направление игровых систем, в котором механика непосредственного управления персонажем сочетается с элементами постепенного развития игровых характеристик в ходе прохождения. Для данного жанра характерно выполнение действий в реальном времени, когда игрок самостоятельно определяет траекторию движения, момент атаки, дистанцию взаимодействия с противником и порядок прохождения игровых участков. В отличие от проектов с заранее фиксированной последовательностью событий, структура прохождения в ActionRPG часто строится вокруг серии локальных боевых эпизодов, между которыми происходит изменение параметров персонажа, получение новых усилений или доступ к следующим игровым зонам. Отдельное развитие внутри жанра получили проекты с элементами roguelite, где после завершения попытки прохождения игровой цикл начинается заново, однако часть накопленного результата сохраняется и используется в последующих попытках. Такой подход позволяет соединить повторяемость базовых механик с постепенным усложнением игровых условий и формирует устойчивую модель многократного прохождения, при которой пользователь постепенно осваивает закономерности игрового пространства и поведения противников.

Игровые программные системы жанра ActionRPG ориентированы на непрерывное взаимодействие пользователя с игровым пространством в реальном времени. Основу игрового процесса составляют перемещение, выбор направления атаки, уклонение от угроз и последовательное прохождение игровых зон. В отличие от проектов, где действия разделены на отдельные фазы, здесь изменение игровой ситуации происходит непрерывно, а результат каждого действия сразу влияет на последующие игровые события. Для двумерных игровых проектов с видом сверху характерно ограниченное игровое пространство, внутри которого игрок одновременно контролирует собственное положение, направление атаки и расстояние до окружающих объектов. При такой организации уровня значительную роль приобретает плотность размещения игровых элементов внутри комнаты, поскольку даже небольшое изменение взаимного расположения объектов влияет на характер столкновения.

Комнатная структура используется как один из устойчивых способов организации игрового процесса в проектах данного жанра. Каждая отдельная зона представляет собой завершённый игровой эпизод с собственным набором препятствий, противников и допустимых направлений движения. После завершения взаимодействия внутри одной комнаты происходит переход к следующему игровому участку, где формируется новая комбинация условий. Такое построение позволяет дозированно изменять интенсивность прохождения и последовательно увеличивать сложность.

Повторяемость игровых действий в пределах одинаковых по структуре

столкновений быстро приводит к формированию устойчивых пользовательских сценариев. Если игровой противник действует по заранее фиксированному набору реакций, игрок после нескольких повторений определяет закономерность поведения и начинает использовать одинаковую последовательность действий в разных игровых ситуациях.

При многократных прохождении это особенно заметно в системах, где игровой цикл строится на последовательном повторении боевых эпизодов. Даже при изменении состава игровых объектов общий характер взаимодействия остается узнаваемым, если отсутствует изменение логики поведения противников. В результате различие между отдельными игровыми ситуациями начинает определяться преимущественно числом объектов на игровом поле.

Отдельное направление развития игровых систем связано с использованием повторных игровых циклов, при которых после завершения попытки пользователь начинает прохождение заново, сохраняя часть накопленного результата. Подобный принцип применяется для постепенного освоения механик и усложнения прохождения без изменения базовой структуры проекта. В таких условиях возрастает значение внутренних игровых алгоритмов, способных обеспечивать различие между повторяющимися игровыми эпизодами.

Дополнительный уровень вариативности достигается за счет изменения конфигурации игрового пространства. Перестроение порядка игровых зон, изменение состава объектов и последовательности переходов создают новые условия взаимодействия даже при ограниченном количестве исходных элементов. Однако изменение расположения объектов не устраняет предсказуемость поведения, если сами игровые реакции остаются неизменными.

В рамках предметной области поведение противников рассматривается как самостоятельный компонент игровой системы, определяющий характер локального столкновения. Противник ограничивает траекторию движения игрока, формирует временные интервалы для атаки и влияет на выбор безопасной позиции в пределах комнаты. При этом поведение должно сохранять понятную внутреннюю логику, чтобы действия игрового объекта воспринимались как закономерные.

Сложность проектирования заключается в том, что повышение интенсивности игрового процесса за счет увеличения числа противников или скорости их движения не приводит к изменению структуры взаимодействия. Игрок продолжает применять знакомую модель действий, адаптируя ее лишь к количеству угроз. По этой причине в исследовании игровых систем жанра ActionRPG внимание уделяется подходам, при которых различие игровых ситуаций достигается изменением выбора поведения внутри уже существующего набора игровых действий.

Такой подход изменяет характер взаимодействия без постоянного расширения количества игровых сущностей и сохраняет устойчивую архитектуру игрового процесса.

1.2 Обзор аналогов

При разработке игровых систем жанра ActionRPG анализ существующих проектов позволяет определить прагматичные решения, применяемые для организации игрового пространства, построения боевых сценариев и формирования повторяемого игрового цикла. В рамках обзора рассматриваются проекты, в которых используются элементы, близкие по структуре к разрабатываемой системе: комнатная организация уровня, последовательное прохождение игровых зон, высокая плотность боевого взаимодействия, повторный игровой цикл и постепенное усложнение поведения противников.

Выбор аналогов обусловлен тем, что в современных игровых проектах данного направления именно сочетание ограниченного пространства, интенсивного взаимодействия с противниками и вариативности игровых ситуаций определяет устойчивость игрового процесса при многократном прохождении. Отдельное значение имеет способ формирования сложности: в ряде проектов она достигается увеличением количества угроз, в других случаях за счет изменения структуры комнаты, темпа боя или набора действий противников.

Анализ аналогов позволяет определить, какие игровые механики оказывают наибольшее влияние на длительность удержания интереса пользователя и какие элементы могут быть использованы при построении собственной игровой модели. Особое внимание уделяется проектам, где повторяемость прохождения сочетается с изменением состава игровых ситуаций, а также присутствует выраженная зависимость между действиями игрока и характером ответной реакции игровых объектов.

1.2.1 The Binding of Isaac

The Binding of Isaac [1] является одним из наиболее показательных примеров построения двумерной игровой системы с комнатной организацией пространства и повторяемым циклом прохождения. Игровой процесс основан на последовательном прохождении замкнутых комнат, соединенных переходами, внутри которых игрок сталкивается с различными типами противников, препятствиями и объектами усиления. Каждая игровая зона представляет собой отдельный боевой эпизод с ограниченным пространством перемещения, где требуется одновременно контролировать направление движения, траекторию атаки и положение угроз.

Структура прохождения строится по принципу последовательного продвижения через этажи подземелья, составленные из случайным образом сформированных комнат. При каждом новом запуске изменяется порядок соединения игровых зон, расположение противников, типы доступных предметов и состав вспомогательных игровых событий. Это обеспечивает значительное число комбинаций даже при ограниченном наборе базовых элементов игрового пространства.

Существенной особенностью проекта является большое количество

игровых предметов, влияющих на характеристики персонажа и характер прохождения. На протяжении длительного периода развития проекта количество доступных предметов было расширено до нескольких сотен, включая пассивные усиления, модификаторы атаки, изменение скорости движения, дополнительные эффекты поражения и специальные игровые свойства. Комбинация нескольких предметов внутри одного прохождения формирует новые варианты поведения игрока и существенно изменяет характер взаимодействия с противниками. Такой объем контента сформировался в результате длительной разработки и многолетней поддержки проекта.

С точки зрения поведения противников игра использует большое количество заранее определенных схем движения и атак. Каждый тип противника действует в пределах собственного набора реакций, связанных с расстоянием до игрока, направлением движения и текущим положением внутри комнаты. Несмотря на разнообразие типов противников, внутренняя логика поведения каждого отдельного объекта остается фиксированной, что позволяет игроку постепенно распознавать повторяющиеся модели.

На рисунке 1.1 представлен игровой процесс The Binding of Isaac.

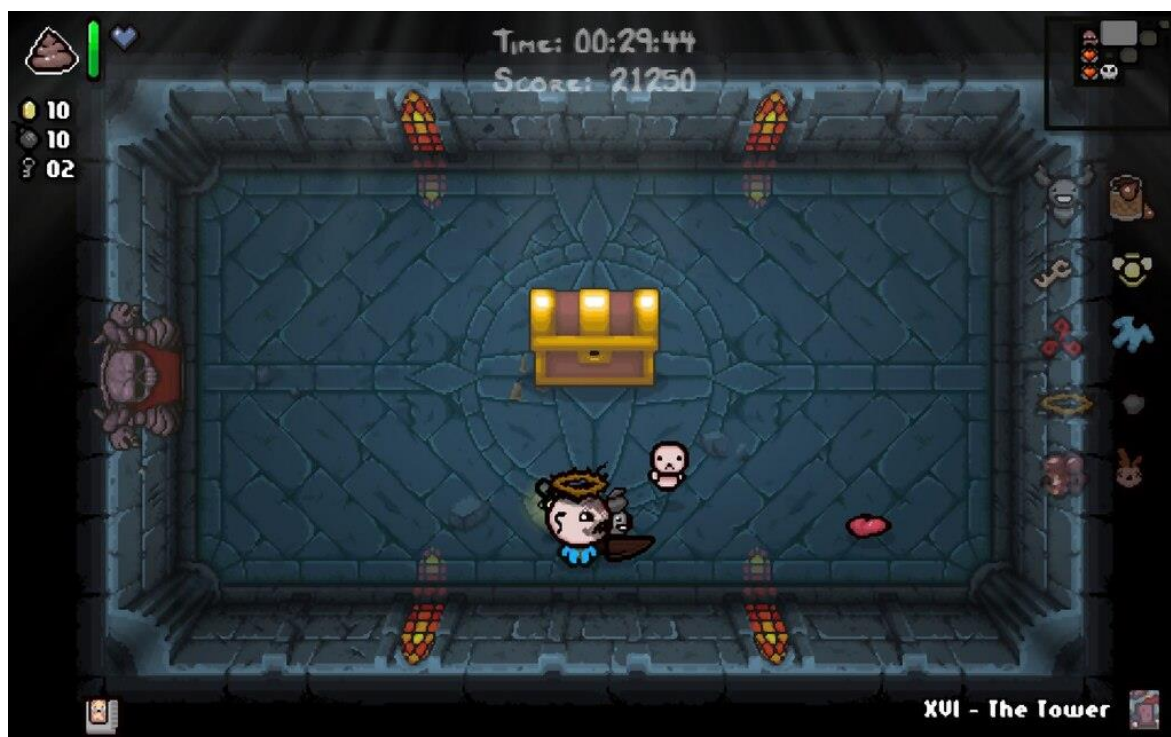


Рисунок 1.1 – Игровой процесс The Binding of Isaac

К преимуществам проекта относятся:

1 Высокая реиграбельность за счет процедурного формирования маршрута прохождения.

2 Большое количество предметов, формирующих различные игровые комбинации.

3 Устойчивая структура коротких боевых эпизодов внутри отдельных

комнат.

4 Значительное разнообразие противников и типов атак.

К недостаткам проекта относятся:

1 Поведение отдельных противников после многократного прохождения становится предсказуемым.

2 Существенная часть сложности формируется через увеличение числа активных угроз.

3 Отдельные игровые комбинации предметов создают резкий дисбаланс сложности.

4 При длительном взаимодействии пользователь начинает использовать заранее сформированные стратегии прохождения.

Данный проект представляет практический интерес для анализа, поскольку демонстрирует устойчивую модель построения игрового цикла, в которой процедурная организация пространства сочетается с большим объемом контента и фиксированными поведенческими шаблонами противников.

1.2.2 Enter the Gungeon

Enter the Gungeon [2] представляет собой двумерный игровой проект, построенный на сочетании комнатной структуры прохождения, высокой плотности боевого взаимодействия и большого количества одновременно активных атакующих объектов на игровом поле. Игровой процесс организован в виде последовательного продвижения по этажам подземелья, где каждая отдельная комната содержит собственный набор противников, препятствий и элементов укрытия.

Структура прохождения построена на случайной генерации маршрута внутри каждого игрового цикла. В игре реализовано несколько сотен единиц вооружения, отличающихся скоростью стрельбы, формой траектории, дополнительными эффектами и взаимодействием с окружающими объектами. Значительная часть сложности формируется не количеством противников, а плотностью траекторий атак, которые одновременно присутствуют в пределах ограниченного пространства комнаты.

Проект развивался в течение продолжительного времени и получил крупные обновления после основного выпуска. Дополнительный контент включал новые игровые зоны, расширенный набор оружия, новых противников и дополнительные игровые события. Последовательное развитие системы позволило существенно увеличить число игровых комбинаций без изменения базовой структуры игрового цикла.

Поведение противников строится на заранее заданных схемах движения и атаки, однако за счет высокой скорости боя и различной геометрии комнат одни и те же типы противников воспринимаются по-разному в разных игровых ситуациях. Многие игровые объекты используют комбинированные траектории движения, резкие смены направления и интервальные атаки, что требует постоянной адаптации действий игрока.

На рисунке 1.2 представлен игровой процесс Enter the Gungeon.



Рисунок 1.2 – Игровой процесс Enter the Gungeon

К преимуществам проекта относятся:

1 Высокая динамика боевых эпизодов в пределах ограниченного пространства.

2 Большое разнообразие оружия и способов взаимодействия с противниками.

3 Использование геометрии комнаты как элемента боевой механики.

4 Процедурное построение игровых маршрутов.

К недостаткам проекта относятся:

1 Основная сложность часто определяется плотностью атак, а не изменением логики поведения противников.

2 При длительном прохождении часть боевых ситуаций начинает восприниматься через узнаваемые схемы движения.

3 Отдельные комбинации вооружения существенно снижают уровень сложности.

4 Высокая интенсивность визуальных эффектов затрудняет восприятие ситуации при большом количестве объектов.

Проект представляет интерес для анализа как пример игровой системы, где вариативность достигается через сочетание процедурного построения пространства, большого числа игровых комбинаций и высокой плотности боевого взаимодействия.

1.2.3 Hades

Hades [3] представляет собой игровой проект, в котором элементы ActionRPG сочетаются с повторяемым циклом прохождения, последовательным усложнением игровых зон и постоянным накоплением игровых улучшений между попытками. Игровой процесс построен вокруг последовательного прохождения отдельных боевых помещений,

объединенных в более крупные этапы, где каждая новая зона отличается составом противников, структурой пространства и характером боевых столкновений.

Система прохождения организована таким образом, что после завершения каждой комнаты игрок получает выбор дальнейшего маршрута и возможных усилений. Это формирует зависимость между краткосрочными решениями внутри текущего прохождения и накоплением преимуществ для последующих этапов.

В отличие от проектов, где основной акцент сделан на количестве предметов, здесь вариативность во многом достигается через управляемую систему выбора улучшений и постепенное изменение набора доступных игровых эффектов.

Поведение противников в игре строится на заранее определенных схемах, однако высокая скорость смены игровых ситуаций и различие геометрии комнат создают ощущение постоянного изменения условий взаимодействия. Многие типы противников используют короткие циклы подготовки атаки, быстрое перемещение и изменение дистанции относительно игрока, что требует постоянного контроля пространства.

Дополнительную роль играет постепенное усложнение игровых зон. На более поздних этапах прохождения возрастает плотность угроз, увеличивается число одновременно активных объектов и изменяется ритм столкновений, что усиливает требования к точности действий игрока.

На рисунке 1.3 представлен игровой процесс Hades.

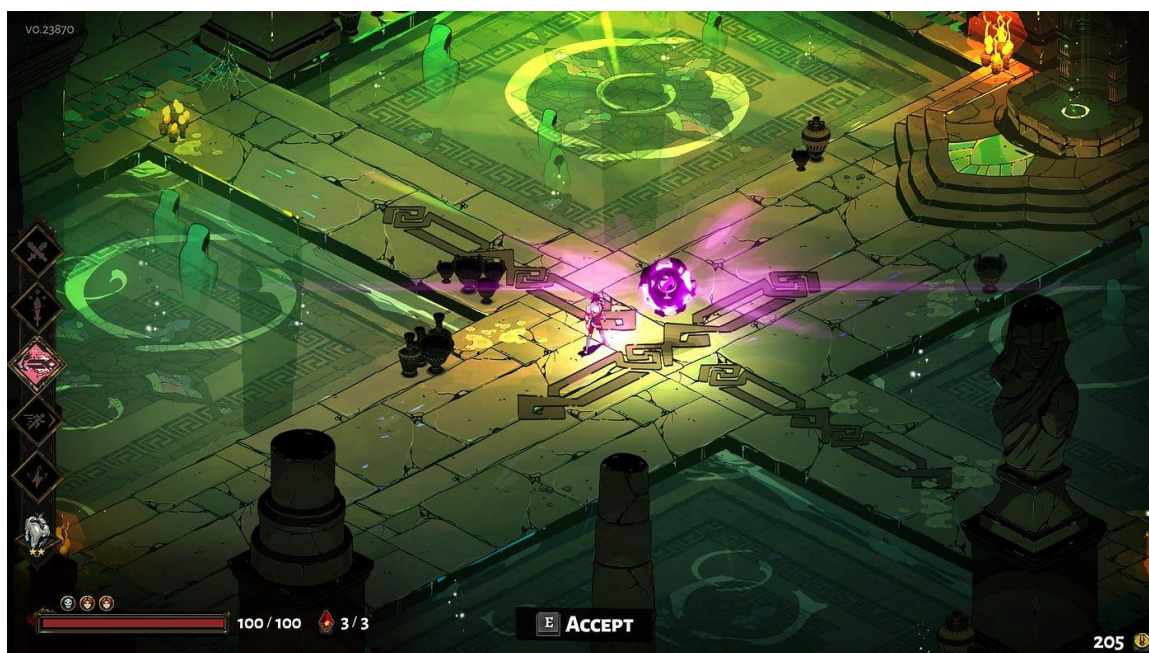


Рисунок 1.3 – Игровой процесс Hades

К преимуществам проекта относятся:

1 Устойчивая система повторного прохождения с накоплением прогресса между игровыми циклами.

2 Большое количество комбинаций временных усиления в пределах одного прохождения.

3 Высокая плотность боевого взаимодействия при сохранении читаемости игрового пространства.

4 Постепенное изменение сложности между игровыми этапами.

К недостаткам проекта относятся:

1 Поведение противников со временем становится узнаваемым.

2 Значительная часть сложности достигается через увеличение интенсивности столкновений.

3 При повторных прохождениях часть игровых решений переходит в устойчивые пользовательские шаблоны.

4 Высокая насыщенность эффектов затрудняет анализ отдельных игровых событий при плотном бою.

Проект представляет интерес для анализа как пример системы, в которой повторяемый игровой цикл сочетается с метапрогрессией, управляемой вариативностью усиления и устойчивой структурой боевых эпизодов.

1.2.4 Dead Cells

Dead Cells [4] представляет собой игровой проект, в котором механика динамичного боевого взаимодействия сочетается с повторяемым циклом прохождения, случайной перестройкой игровых зон и постепенным накоплением постоянных улучшений между игровыми попытками. Несмотря на то что проект выполнен в формате двумерного платформенного пространства, его структура по организации игрового цикла и управлению сложностью близка к системам жанра roguelite.

Игровое прохождение строится вокруг последовательного перемещения через взаимосвязанные игровые зоны, содержащие противников, боевые участки, скрытые маршруты и элементы усиления.

Особенность проекта связана с тем, что скорость игрового взаимодействия сохраняется высокой на протяжении всего прохождения. Большая часть боевых эпизодов требует постоянного перемещения, точного выбора момента атаки и контроля безопасной дистанции. При этом система движения и боя построена так, что игрок постоянно находится в режиме активного принятия решений.

Поведение противников строится на заранее заданных паттернах движения и атак, однако большое значение имеет различие скоростей отдельных игровых объектов и необходимость учитывать особенности окружающего пространства.

Многие противники используют резкие смены положения, быстрый выход на дистанцию удара и короткие интервалы между атаками, что требует от игрока постоянного анализа ближайшей игровой ситуации. Игрок самостоятельно выбирает направление дальнейшего продвижения, что влияет на состав противников, типы игровых зон и интенсивность последующих столкновений.

На рисунке 1.4 представлен игровой процесс Dead Cells.

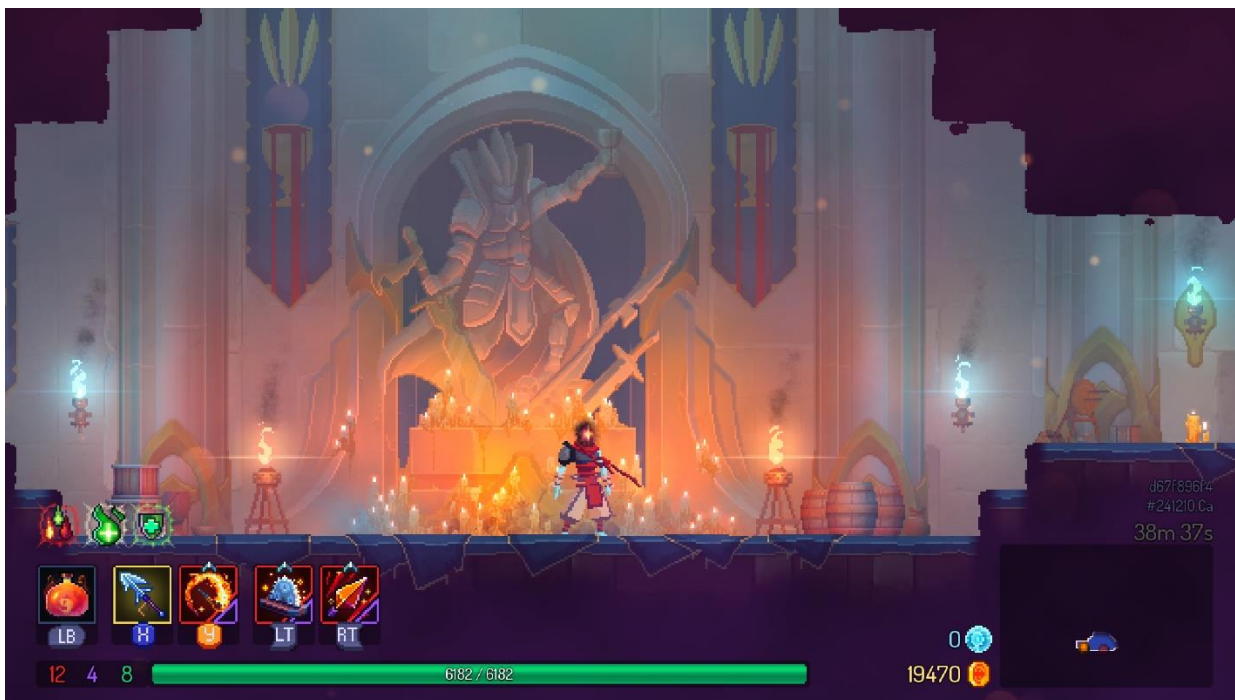


Рисунок 1.4 – Игровой процесс Dead Cells

К преимуществам проекта относятся:

- 1 Высокая скорость игрового процесса и устойчивый темп боевых эпизодов.

- 2 Большое количество комбинаций оружия и активных игровых навыков.

- 3 Процедурное изменение маршрутов прохождения.

К недостаткам проекта относятся:

- 1 Поведение противников после длительного прохождения становится предсказуемым внутри отдельных игровых зон.

- 2 Отдельные игровые комбинации вооружения снижают необходимость адаптации к поведению противников.

- 3 При повторных прохождениях пользователь начинает использовать устойчивые маршруты и заранее сформированные решения.

Проект представляет интерес для анализа как пример игровой системы, где высокая динамика прохождения сочетается с перестройкой игровых условий и накоплением игровых преимуществ между попытками.

1.2.5 Moonlighter

Moonlighter [5] представляет собой игровой проект, сочетающий элементы двумерного боевого прохождения подземелий и систему развития между игровыми циклами через внешнюю экономическую модель. Игровой процесс разделен на две взаимосвязанные части: прохождение боевых зон с последовательным исследованием подземелья и управление накопленными ресурсами вне активного игрового цикла.

Основная часть прохождения строится вокруг исследования подземелий, состоящих из отдельных комнат, соединенных переходами.

Внутри каждой комнаты размещаются противники, объекты взаимодействия и награды, получаемые после завершения боевого эпизода. После очистки зоны игрок получает возможность продолжить продвижение вглубь уровня или завершить текущую попытку с сохранением найденных ресурсов.

Особенностью проекта является сохранение результатов между игровыми циклами через систему развития внешней игровой среды. Полученные в подземелье ресурсы используются для расширения возможностей персонажа, улучшения доступного снаряжения и постепенного увеличения эффективности следующих прохождений. Такое построение связывает отдельные игровые попытки в единую последовательность развития.

Поведение противников построено на фиксированных схемах движения и атак, связанных с конкретным типом игрового объекта. Каждый тип противника использует ограниченный набор реакций, который определяется дистанцией до игрока и текущим положением внутри комнаты. При длительном взаимодействии пользователь постепенно формирует устойчивое понимание этих схем и заранее подбирает безопасную модель прохождения.

На рисунке 1.5 представлен игровой процесс Moonlighter.

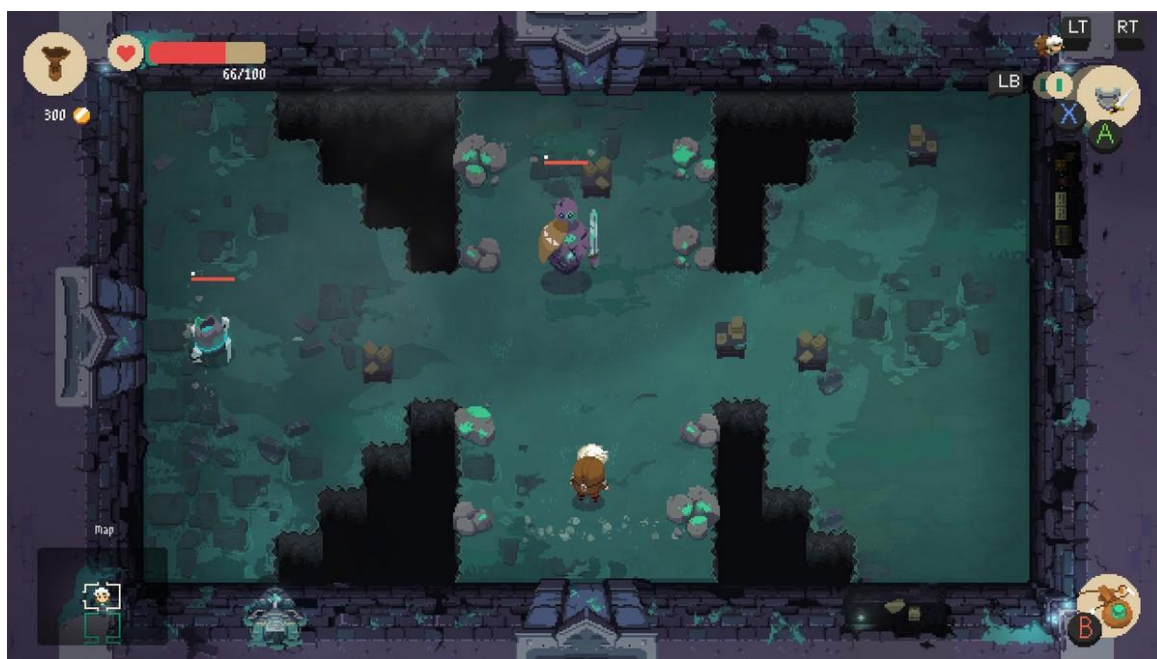


Рисунок 1.5 – Игровой процесс Moonlighter

К преимуществам проекта относятся:

- 1 Сочетание боевого прохождения и системы накопления прогресса между игровыми циклами.
- 2 Процедурное изменение структуры подземелий.
- 3 Разнообразие экипировки и способов ведения боя.
- 4 Устойчивая связь между отдельными игровыми попытками через систему развития.

К недостаткам проекта относятся:

1 Поведение противников внутри отдельных типов быстро становится узнаваемым.

2 Основная сложность часто определяется характеристиками экипировки.

3 При повторных прохождении часть игровых ситуаций начинает восприниматься как заранее известная последовательность действий.

4 Ограниченное число базовых поведенческих моделей снижает различие между поздними игровыми этапами.

Проект представляет интерес для анализа как пример игровой системы, где процедурная организация пространства сочетается с метапрогрессией и длительным накоплением игровых преимуществ между повторными прохождениями.

1.3 Обзор средств разработки

Разработка игровых программных систем требует выбора инструментов, обеспечивающих одновременно построение игровой логики, управление визуальными объектами, организацию взаимодействия игровых сущностей и поддержку последующего расширения проекта. При выборе средств разработки учитываются особенности жанра, предполагаемый объем программной части, требования к производительности и доступность инструментов для интеграции различных компонентов в единую систему.

Для игровых проектов жанра ActionRPG существенное значение имеет возможность разделения исполнительской части и логических модулей. Игровая система включает обработку пользовательских действий, управление игровыми объектами, построение боевых сценариев, работу с визуальными эффектами и сохранение данных между игровыми циклами. Это требует использования средств, позволяющих сочетать высокоуровневое управление сценой с возможностью реализации отдельных алгоритмических компонентов на уровне исходного кода.

В современной игровой разработке используется широкий спектр языков программирования и специализированных программных платформ. Их выбор определяется требованиями конкретного проекта, типом игрового движка, моделью хранения данных и способом построения пользовательского взаимодействия. Для двумерных игровых систем с внутренней модульной логикой особое значение приобретают инструменты, позволяющие реализовывать собственные алгоритмы поведения без ограничения встроенными шаблонами.

1.3.1 Языки программирования

C++ [6] относится к числу основных языков, применяемых при разработке компьютерных игр и высокопроизводительных интерактивных систем. Язык сочетает объектно-ориентированный подход, возможность работы с низкоуровневыми структурами памяти и высокую скорость выполнения программного кода после компиляции. За счет прямого

управления ресурсами и широких возможностей оптимизации C++ используется в игровых движках, системах обработки графики, физическом моделировании и построении внутренних игровых алгоритмов.

В игровой разработке C++ широко применяется для создания модулей, работающих в условиях постоянного обновления игровых состояний, где каждая вычислительная операция выполняется многократно в течение игрового цикла. Это относится к системам обработки столкновений, расчета поведения игровых объектов, обновления координат, контроля состояний и внутреннего обмена данными между компонентами проекта. При построении игровых модулей такой подход позволяет сохранять предсказуемость структуры программы и гибкость в проектировании архитектуры.

Дополнительным преимуществом языка является возможность построения собственной иерархии классов и логического разделения игровых сущностей. В игровых проектах это позволяет выделять отдельные программные компоненты для управления персонажами, противниками, обработкой игровых событий и вычислительными алгоритмами. При использовании C++ разработчик получает доступ к внутренней структуре игрового движка и может реализовывать логику, не ограничиваясь только встроенными инструментами визуального программирования.

C# [7] широко применяется в игровой разработке благодаря сочетанию объектно-ориентированной модели, автоматического управления памятью и развитой стандартной библиотеки. Наиболее активно язык используется в проектах, создаваемых на движке Unity, где он является основным средством реализации игровой логики. C# обеспечивает удобное построение игровых сценариев, управление объектами сцены, обработку пользовательского ввода и создание интерфейсных компонентов.

Особенность языка заключается в более высоком уровне абстракции по сравнению с C++. Это упрощает создание игровых систем среднего масштаба и сокращает количество низкоуровневых операций. При этом часть вычислительных возможностей ограничивается внутренней архитектурой среды выполнения, что в крупных проектах требует более внимательного подхода к производительности.

Java [8] сохраняет устойчивое применение в проектах, где требуется кроссплатформенная серверная часть, распределенная обработка данных или интеграция игровых сервисов с внешними программными системами. Язык обладает развитой системой стандартных библиотек, строгой типизацией и стабильной моделью выполнения через виртуальную машину.

В игровой индустрии Java используется значительно реже для клиентской части игровых проектов, однако сохраняет значение в многопользовательских сервисах, серверных приложениях и системах обработки пользовательских данных. Его применение оправдано в случаях, когда основная нагрузка сосредоточена вне графической части проекта.

1.3.2 Среда разработки

Unreal Engine [9] относится к числу наиболее развитых игровых

движков, применяемых при создании интерактивных систем различной сложности. Среда объединяет инструменты построения игровых сцен, систему работы с объектами, управление коллизиями, анимацией, визуальными эффектами и внутренней логикой проекта. Разработка внутри движка строится вокруг единой структуры проекта, где игровые объекты, уровни, визуальные ресурсы и программные классы связаны общей архитектурой.

Существенным преимуществом Unreal Engine является компонентный принцип построения игровых объектов. Каждый игровой объект может включать набор независимых компонентов, отвечающих за отображение, движение, обработку столкновений, получение событий и взаимодействие с другими объектами сцены. Такой подход позволяет формировать сложные игровые сущности без нарушения общей структуры проекта и обеспечивает удобное разделение отдельных функциональных частей.

Для двумерных игровых проектов в составе движка используется подсистема Paper2D, предназначенная для работы со спрайтами, покадровой анимацией, тайловыми картами и двумерными игровыми сценами. Подсистема позволяет формировать игровое пространство на основе тайловых элементов, управлять анимационными состояниями игровых объектов и создавать уровни с фиксированной геометрией или комнатной структурой.

Существует встроенная система визуальных эффектов Niagara, позволяющая формировать графические события, связанные с игровыми столкновениями, уничтожением объектов и реакцией на действия игрока. Визуальные эффекты внутри движка интегрируются непосредственно в игровую логику и могут запускаться как реакция на изменение состояния игровых объектов.

Для крупных игровых проектов использование Unreal Engine связано с высокими требованиями к вычислительным ресурсам рабочей станции и значительным объемом самого проекта. Даже при разработке двумерных игровых систем среда использует внутреннюю инфраструктуру, рассчитанную на сложные трехмерные сцены, что увеличивает объем служебных файлов и время компиляции. Полноценное применение движка наиболее оправдано в проектах, где требуется построение собственной программной архитектуры, работа с внутренними игровыми подсистемами и возможность расширения базовых механизмов. Наиболее активно Unreal Engine применяется при разработке ActionRPG, шутеров, проектов с развитой системой состояний игровых объектов и сложной внутренней логикой.

Unity [10] представляет собой широко распространенную игровую среду, используемую для разработки двухмерных и трехмерных приложений различного масштаба. В основе работы среды лежит компонентная модель построения игровых объектов, при которой каждый объект сцены получает набор подключаемых функциональных элементов, отвечающих за визуальное представление, физическое поведение, обработку событий и пользовательское взаимодействие.

Среда включает визуальный редактор сцен, систему анимации, встроенный физический движок, средства работы со звуком, интерфейсом и

ресурсами проекта. Основная логика реализуется на языке C#, который тесно интегрирован с внутренней структурой движка и используется для описания поведения игровых объектов, управления сценой и обработки пользовательских действий.

При этом в проектах, где требуется глубокая реализация собственных алгоритмов и высокая степень контроля над внутренними вычислениями, возможности среды ограничиваются архитектурой управляемого выполнения и структурой встроенных компонентов.

Unity сохраняет высокую эффективность при разработке мобильных игр, независимых проектов, двумерных платформеров, аркадных приложений и игровых систем среднего масштаба. Ограничения среды становятся заметны в проектах, где требуется глубокое изменение внутренних механизмов работы движка или высокая степень контроля над вычислительной частью. При увеличении числа одновременно активных игровых объектов часть задач требует дополнительной оптимизации. В крупных проектах архитектура управляемого выполнения может создавать ограничения при построении сложных вычислительных модулей.

Godot [11] представляет собой свободную игровую среду с открытым исходным кодом, предназначенную для разработки двумерных и трехмерных интерактивных систем. В основе движка используется собственная архитектура узлов, где каждый игровой объект строится как иерархия взаимосвязанных элементов, отвечающих за отдельные части поведения и отображения.

Среда включает редактор сцен, систему анимации, обработку коллизий, встроенные средства работы с тайловыми картами и визуальными эффектами. Для программирования используется язык GDScript, синтаксически близкий к Python, а также поддерживаются C# и дополнительные механизмы расширения через нативные модули.

Открытая архитектура дает возможность изменять внутренние части движка и адаптировать отдельные механизмы под задачи конкретного проекта. Однако при разработке крупных игровых систем количество готовых встроенных инструментов уступает более крупным коммерческим решениям.

Godot наиболее часто используется при создании небольших двумерных проектов, обучающих игровых приложений и независимых игровых прототипов. Ограничением среды остается сравнительно меньшее количество готовых профессиональных инструментов и библиотек по сравнению с коммерческими игровыми платформами. При построении крупных игровых проектов часть специализированных задач требует самостоятельной доработки или подключения дополнительных решений. На практике движок чаще применяется в проектах с ограниченной командой разработки и относительно компактной архитектурой.

Visual Studio [12] представляет собой интегрированную среду разработки, ориентированную на создание программных систем на языках C++, C#, Python и ряде других языков. Среда включает редактор исходного кода, систему интеллектуального автодополнения, средства навигации по

проекту, встроенный отладчик и инструменты анализа структуры программного кода.

При разработке игровых проектов Visual Studio используется для построения программной части игровых систем, включая создание игровых классов, модулей логики поведения, обработку игровых состояний и реализацию внутренних вычислительных алгоритмов. Среда обеспечивает прямую работу с программными файлами игровых проектов и поддерживает крупные кодовые базы.

Существует развитая система отладки, позволяющая пошагово анализировать выполнение программы, отслеживать значения переменных, состояние объектов и последовательность вызова функций.

Visual Studio активно используется при разработке игровых проектов, в которых основная часть логики реализуется на C++ или C#. Среда особенно востребована в проектах, создаваемых на Unreal Engine, а также при разработке собственных игровых модулей, серверных частей и вспомогательных инструментов сопровождения. Ограничением среды является значительное количество внутренних настроек и высокая зависимость удобства работы от корректной конфигурации проекта. При крупных игровых решениях время индексирования и анализа проекта возрастает вместе с объемом исходного кода.

1.3.3 Система контроля версий и хостинг

Git [13] представляет собой распределенную систему контроля версий, предназначенную для фиксации изменений в исходном коде, отслеживания последовательности разработки и управления состоянием проекта на различных этапах его формирования. Основным принцип работы Git заключается в сохранении набора изменений в виде последовательных фиксаций, каждая из которых содержит информацию о состоянии проекта в конкретный момент времени.

Использование системы контроля версий имеет особое значение при разработке программных проектов, содержащих большое количество взаимосвязанных файлов, поскольку позволяет безопасно вносить изменения в архитектуру проекта, возвращаться к предыдущим состояниям и анализировать историю изменений. В игровых системах это особенно важно при параллельной работе с программной логикой, ресурсами проекта и внутренней структурой игровых модулей.

Одним из базовых механизмов Git является работа с ветвями разработки. Ветвление позволяет создавать отдельные направления изменения проекта, тестировать новые решения без влияния на основную рабочую версию и объединять завершенные изменения после проверки результата. Такой подход позволяет изолировать экспериментальные изменения и сохранять стабильность основной версии проекта.

В программной разработке Git применяется практически во всех типах проектов: от небольших учебных решений до крупных игровых и промышленных систем, где требуется постоянный контроль изменения

исходного кода и возможность возврата к стабильным состояниям проекта.

GitHub [14] представляет собой веб-платформу для размещения репозитория Git и удаленного хранения исходного кода. Платформа обеспечивает централизованное хранение проекта, доступ к истории изменений, управление версиями и возможность резервного копирования исходных файлов.

Использование GitHub позволяет хранить проект вне локальной среды разработки, синхронизировать изменения между рабочими устройствами и сохранять историю разработки независимо от состояния локального компьютера. Платформа также предоставляет инструменты просмотра истории изменений, анализа различий между версиями и управления отдельными ветвями проекта.

В программных проектах GitHub часто используется как средство подтверждения авторского вклада, поскольку каждая операция фиксации сопровождается временной отметкой и сохраняет последовательность внесенных изменений.

GitHub широко используется в индивидуальной и командной разработке программных систем, включая игровые проекты, веб-приложения, библиотеки и исследовательские программные решения. Ограничением платформы является зависимость от внешней облачной инфраструктуры, поскольку доступ к репозиторию и синхронизация изменений требуют подключения к сети. При работе с крупными игровыми проектами значительный объем бинарных файлов усложняет хранение ресурсов и требует дополнительной организации структуры репозитория. В практической разработке сервис распространен в независимых проектах, учебных работах, открытых программных решениях и распределенных командах.

GitLab [15] представляет собой платформу управления репозиториями, основанную на системе Git и дополненную средствами организации процессов сопровождения программного проекта. Помимо хранения исходного кода платформа включает инструменты контроля задач, отслеживания изменений, автоматической сборки и проверки состояния проекта.

Одной из особенностей GitLab является расширенная система управления этапами разработки, позволяющая связывать изменения в коде с конкретными задачами проекта. Это обеспечивает более детальную организацию крупных программных решений, где отдельные изменения должны фиксироваться как часть более общей структуры разработки.

Платформа применяется в проектах, где требуется более детальное управление процессом разработки и объединение хранения исходного кода с внутренней организацией задач. Ограничением является более сложная настройка отдельных механизмов сопровождения по сравнению с более простыми сервисами удаленного хранения. При локальном развертывании возрастает нагрузка на сопровождение собственной серверной инфраструктуры. В реальной разработке часто используется в корпоративных программных системах, внутренних инженерных проектах и командах, где требуется контроль полного цикла изменения программного продукта.

1.4 База данных

При проектировании программных систем выбор способа хранения данных определяется характером информации, частотой обращения к ней, требованиями к скорости доступа и объемом сохраняемых состояний. В игровых проектах база данных используется преимущественно для хранения параметров, которые должны сохраняться между игровыми циклами: пользовательские настройки, накопленный прогресс, статистические показатели, открытые улучшения и дополнительные игровые параметры.

Для игровых систем с локальным выполнением основной логики большая часть игровых состояний обрабатывается в оперативной памяти, поскольку игровые объекты, столкновения, вычисление состояний и поведение противников должны обновляться в реальном времени без обращения к внешнему хранилищу данных. База данных используется только для тех компонентов, которые сохраняются между отдельными игровыми запусками и должны быть доступны после завершения текущего игрового цикла.

SQLite [16] представляет собой встроенную реляционную базу данных, не требующую отдельного серверного процесса для работы. Вся структура данных хранится в одном локальном файле, который подключается непосредственно программой во время выполнения. Работа с данными осуществляется через стандартные SQL-запросы, что позволяет использовать привычную реляционную модель хранения без развертывания дополнительной серверной инфраструктуры.

База данных отличается минимальными требованиями к ресурсам и простотой интеграции в программный проект. Отсутствие отдельного сервера упрощает сопровождение базы данных и позволяет использовать ее как часть локальной структуры приложения. Для программных решений, где объем сохраняемых данных ограничен и отсутствует необходимость одновременного доступа большого числа пользователей, такая модель хранения является практичной.

В игровых системах широко применяется для хранения пользовательских настроек, параметров сохранения, статистики прохождений, данных метапрогрессии и локальных таблиц параметров. При этом текущая игровая логика обычно не взаимодействует с базой данных в режиме постоянного обновления, а обращение выполняется только в моменты сохранения или загрузки.

SQLite широко используется в игровых приложениях, где требуется локальное хранение ограниченного объема информации без построения отдельной серверной инфраструктуры. На практике база данных применяется в одиночных играх, мобильных приложениях, локальных системах сохранения, хранении параметров прохождения, игровых настроек и таблиц метапрогрессии.

Ограничением SQLite является отсутствие эффективной работы при большом количестве одновременных операций записи и невозможность

масштабирования под многопользовательскую нагрузку. Для игровых проектов, где основная часть логики выполняется локально, такие ограничения остаются допустимыми.

PostgreSQL [17] представляет собой объектно-реляционную систему управления базами данных, ориентированную на сложные структуры хранения и расширенные возможности обработки информации. Система поддерживает стандартные SQL-операции, транзакции, сложные типы данных, вложенные структуры и механизмы расширения через дополнительные модули.

База данных используется в проектах, где требуется работа со сложными взаимосвязанными данными, высокий уровень согласованности и возможность масштабирования под значительную вычислительную нагрузку. База данных обеспечивает устойчивую обработку больших массивов информации и поддерживает расширенные механизмы анализа данных.

При разработке игровых систем применяется преимущественно в серверной части многопользовательских проектов, системах аналитики, хранении больших объемов пользовательских данных и централизованной обработке статистики.

PostgreSQL используется в игровых системах, где требуется хранение сложных взаимосвязанных данных и работа с большими объемами серверной информации. В реальной практике база данных применяется в многопользовательских игровых платформах, аналитических игровых сервисах, системах обработки телеметрии и хранении больших игровых журналов.

Ограничением является более высокая сложность администрирования по сравнению с более легкими решениями. Для локальных игровых проектов использование такой базы данных требует ресурсов, не оправданных объемом сохраняемой информации.

MongoDB [18] представляет собой нереляционную базу данных, в которой информация хранится в виде документов с вложенной структурой. Данные организуются в JSON-подобном формате, что позволяет сохранять объекты без жесткой схемы таблиц и легко изменять внутреннюю структуру записей.

База данных используется в системах, где структура данных изменяется в процессе развития проекта или заранее не определяется полностью. Гибкость хранения позволяет работать с различными типами информации без предварительного формирования жесткой реляционной модели.

В программной разработке MongoDB применяется в высоконагруженных веб-сервисах, облачных системах, приложениях с динамическими пользовательскими данными и распределенных сервисах.

Для игровых проектов база данных может использоваться при хранении телеметрии, событий больших игровых сессий, пользовательских журналов и данных, не требующих строгих реляционных связей.

Ограничением MongoDB является отсутствие жесткой реляционной структуры, что усложняет контроль целостности данных при хранении параметров,

связанных строгими зависимостями. Для игровых проектов с небольшой локальной системой сохранения применение документной модели хранения часто оказывается избыточным. Когда требуется точная структура хранения и устойчивые связи между наборами параметров, требуется дополнительный контроль схемы данных.

База данных применяется в игровых проектах, связанных с обработкой больших объемов событийных данных, хранением телеметрии, пользовательской активности и динамических игровых журналов.

Ограничением MongoDB является отсутствие жесткой реляционной структуры, что усложняет контроль целостности данных при хранении параметров, связанных строгими зависимостями. Для игровых проектов с небольшой локальной системой сохранения применение документной модели хранения часто оказывается избыточным.

1.5 Концепция разрабатываемой игровой системы

Разрабатываемая игровая система относится к двумерным проектам жанра ActionRPG с последовательным прохождением игровых зон, повторяемым циклом попыток и постепенным накоплением игровых преимуществ между отдельными прохождениями. В основе игровой структуры используется модель продвижения через отдельные комнаты подземелья, внутри которых формируются локальные боевые эпизоды с ограниченным пространством взаимодействия.

Игровое пространство организовано как последовательность взаимосвязанных комнат, соединенных переходами. Каждая игровая зона содержит собственный набор противников, препятствий и доступных направлений перемещения. После завершения взаимодействия внутри одной комнаты открывается возможность перехода к следующему участку подземелья.

Сюжетная основа проекта связана с подземельем, расположенным под руинами древней крепости, где скрывается маг-лич, ранее занимавший положение придворного колдуна и изгнанный за проведение экспериментов с магией времени и иллюзий. После изгнания он продолжил исследования в изолированном пространстве, где сформировал магическую область с нестабильной внутренней структурой

В пределах созданного пространства комнаты подземелья постоянно изменяют расположение, переходы между зонами перестраиваются, а внутренняя структура игрового маршрута не сохраняется между отдельными попытками прохождения. Это создает сюжетное объяснение процедурного формирования игрового пространства и позволяет связать техническую организацию уровня с внутренней логикой игрового мира

Игровой персонаж представлен рыцарем, направленным в подземелье по приказу правителя королевства для устранения источника угрозы. На раннем этапе прохождения становится ясно, что обычное продвижение внутрь подземелья не приводит к завершению задачи, поскольку после гибели

персонаж возвращается к начальному моменту игрового цикла.

Повторяемость игровых попыток объясняется действием временной аномалии, возникшей как следствие магических экспериментов внутри подземелья. После каждой неудачной попытки прохождения персонаж сохраняет часть накопленного опыта, полученных усилений и знаний о внутренней структуре игровых ситуаций. Это формирует устойчивую модель повторного игрового цикла, в которой каждая новая попытка использует результат предыдущего прохождения

Такое построение игровой концепции позволяет связать сюжетную основу, организацию игрового пространства и внутренние механизмы изменения сложности в пределах единой модели игрового процесса.

1.6 Выводы

На основании проведенного обзора предметной области, анализа существующих игровых решений и рассмотрения используемых средств разработки сформированы основные положения, определяющие дальнейшее проектирование дипломного проекта.

Исследование предметной области показало, что для проектов жанра ActionRPG устойчивый интерес представляет сочетание ограниченного игрового пространства, повторяемого цикла прохождения и постепенного усложнения игровых ситуаций. Комнатная организация уровня позволяет формировать локальные боевые эпизоды с контролируемым набором условий. При повторных прохождениях значение приобретает не только изменение структуры пространства, но и сохранение различий между отдельными столкновениями за счет изменения логики игровых объектов.

Анализ существующих проектов показал, что современные решения жанра ActionRPG обеспечивают высокую реиграбельность за счет процедурного изменения маршрута прохождения, расширенного набора предметов и вариативности вооружения. При этом даже в проектах с большим количеством игровых комбинаций поведение противников внутри отдельных типов часто строится на фиксированных паттернах, которые после многократного взаимодействия становятся узнаваемыми.

Рассмотренные аналоги показывают, что в игровых системах с повторяемой структурой прохождения усложнение нередко достигается увеличением числа противников, ростом плотности атак или расширением набора игровых объектов. В большинстве решений вариативность достигается за счет расширения набора предметов, усложнения конфигурации игровых зон, увеличения числа противников и введения дополнительных механик усиления персонажа. При этом внутренняя логика поведения обычно ограничивается заранее заданными схемами движения и фиксированными условиями атаки. В разрабатываемом проекте основное отличие заключается в выделении отдельного программного модуля адаптивного поведения как самостоятельного объекта разработки, где изменение реакции противника определяется совокупностью параметров игровой ситуации, включая

дистанцию до игрока, текущее состояние противника, плотность игрового пространства и сохраненные данные о последних действиях игрока.

Проведенный обзор языков программирования подтвердил, что для реализации алгоритмической части игровых систем, требующей постоянного обновления состояний и управления внутренней логикой игровых объектов, наибольшую практическую применимость сохраняет язык C++. Его использование обеспечивает построение собственной архитектуры классов, реализацию конечного автомата состояний и вычисление параметров выбора поведения в пределах игрового цикла.

В качестве основной среды разработки выбран Unreal Engine, поскольку движок предоставляет готовую инфраструктуру для организации игровых сцен, управления объектами, обработки столкновений и построения двумерной игровой среды. Для программной реализации модулей поведения применяется среда Visual Studio, обеспечивающая работу с исходным кодом проекта и отладку игровых классов.

Для сопровождения проекта и фиксации этапов разработки используется Git как система контроля версий и GitHub как средство удаленного хранения исходного кода. Применение системы контроля версий позволяет последовательно фиксировать этапы формирования программного модуля, сохранять историю изменений и подтверждать самостоятельность разработки.

Для хранения данных между игровыми циклами выбрана SQLite, поскольку структура сохраняемой информации в проекте ограничивается параметрами метапрогрессии, пользовательскими настройками и статистикой прохождений. Выбор соответствует архитектуре одиночной игровой системы, где основная обработка игровых состояний выполняется в оперативной памяти, а обращение к базе данных требуется только при сохранении и загрузке данных.

Принятая игровая концепция позволяет использовать игровую систему как среду функционирования программного модуля адаптивного поведения.

В качестве средств разработки были выбраны:

- для реализации программного модуля поведения и основной игровой логики: язык программирования C++ и среда разработки Visual Studio;
- для формирования игровой среды и организации взаимодействия игровых объектов: Unreal Engine;
- для хранения параметров метапрогрессии, игровых улучшений и статистики между игровыми циклами: SQLite;
- для контроля этапов разработки и фиксации изменений исходного кода Git и GitHub.

Выбранные технологические решения соответствуют поставленной цели дипломного проекта и обеспечивают переход к этапу проектирования архитектуры программного модуля и реализации игровой системы.

2 СИСТЕМНОЕ ПРОЕКТИРОВАНИЕ

Требуется определить общую структуру программной системы и выделить функциональные компоненты, обеспечивающие работу игрового приложения и программного модуля адаптивного поведения противников. Поскольку объектом разработки является не только отдельный программный модуль адаптивного поведения, но и игровая среда, архитектура проекта должна учитывать разделение между исполнительной частью игрового процесса и логикой принятия решений внутри игровых объектов.

Разрабатываемая система представляет собой двумерное игровое приложение жанра ActionRPG, в котором игровая среда используется как пространство функционирования программного модуля адаптивного поведения. Игровой цикл включает обработку пользовательского ввода, управление игровым персонажем, формирование боевых столкновений, переход между комнатами, изменение игровых состояний и сохранение части данных между отдельными прохождениями.

Для построения архитектуры проекта выделяются основные функциональные блоки:

- блок обработки пользовательского ввода;
- блок управления персонажем;
- блок генерации уровня;
- блок боевой системы;
- блок адаптивного поведения противников;
- блок управления игровым состоянием;
- блок пользовательского интерфейса;
- блок сохранения и метапрогрессии;
- блок отображения игры.

Работа программной системы организуется следующим образом:

- пользовательские действия поступают в блок обработки ввода;
- команды движения и игровых действий передаются в блок управления персонажем;
- команды системного характера передаются в блок управления игровым состоянием;
- перемещение и атака игрока изменяют состояние боевой системы;
- блок боевой системы взаимодействует с блоком поведения противников;
- блок адаптивного поведения получает игровые параметры и определяет текущее состояние противника;
- блок управления игровым состоянием контролирует переход между комнатами, завершение игровых эпизодов и изменение стадий прохождения;
- блок генерации уровня формирует структуру игровых комнат;
- блок пользовательского интерфейса отображает параметры состояния игрока и текущие игровые показатели;
- блок сохранения фиксирует накопленные параметры между игровыми циклами;

– блок отображения обеспечивает визуальное представление игровой сцены.

Взаимодействие между функциональными блоками отражается в общей структурной схеме ГУИР.400201.112 С1, где центральное положение, на ряду с блоком управления игровым состоянием, занимает блок адаптивного поведения противников как основной объект программной разработки.

2.1 Обоснование выбора средств разработки

Выбор средств разработки определяется требованиями к архитектуре программной системы, характером вычислительной нагрузки и необходимостью разделения игровой среды и программного модуля принятия решений. Поскольку в рамках дипломного проекта разрабатывается не только игровое приложение, но и отдельный алгоритм адаптивного поведения противников, используемые инструменты должны обеспечивать возможность реализации собственной логики состояний, управления игровыми объектами и сопровождения исходного кода на уровне программных классов.

При выборе технологической основы учитывались несколько критериев: поддержка двумерного игрового пространства, возможность построения модульной структуры проекта, работа с покадровой анимацией, доступ к внутренней программной архитектуре и наличие инструментов отладки. Отдельное внимание уделялось возможности разделения вычислительной логики и исполнительской части игровых объектов, поскольку основной объект разработки связан с построением самостоятельного программного модуля поведения.

Дополнительным критерием являлась возможность локального хранения данных между игровыми циклами без усложнения общей структуры проекта серверной инфраструктурой. Для сопровождения разработки также учитывалась необходимость фиксации изменений исходного кода и сохранения последовательности формирования программной части проекта.

2.2 Язык программирования C++

C++ является одним из базовых языков программирования, применяемых при разработке высокопроизводительных программных систем, где требуется постоянная обработка большого количества состояний и контроль над внутренней структурой выполнения. Язык сочетает объектно-ориентированные механизмы проектирования, работу с памятью и высокую скорость выполнения после компиляции, благодаря чему сохраняет устойчивое применение в игровой индустрии и инженерных приложениях.

В игровой разработке C++ используется при построении игровых движков, реализации игровых сущностей, обработке столкновений и вычислительных модулей, работающих в пределах каждого кадра игрового цикла. В отличие от языков, использующих управляемую среду выполнения, C++ позволяет напрямую контролировать жизненный цикл игровых объектов

и порядок обновления логики.

В рамках данного дипломного проекта применение C++ связано с разработкой отдельного модуля адаптивного поведения противников. Модуль должен функционировать независимо от встроенных шаблонов движка и определять состояние противника на основе параметров игровой ситуации: расстояния до игрока, состояния противника, ближайших объектов, недавних действий игрока и условий игровой комнаты.

Состояния поведения противника, включая ожидание, преследование, атаку, отход и обходное перемещение, оформляются как система переходов между состояниями. Каждый переход определяется набором условий и вычисляемых параметров. Использование C++ позволяет организовать такую структуру через отдельные классы и методы обработки состояний.

Основные преимущества использования C++ в рамках проекта:

1 Высокая скорость выполнения вычислений, необходимая для многократного выполнения алгоритма поведения в игровом цикле.

2 Возможность построения собственной объектной архитектуры для противников, вычислительного блока угрозы и FSM.

3 Прямая работа с игровыми классами Unreal Engine.

4 Точное управление внутренним состоянием игровых объектов.

Еще одним преимуществом является возможность отделить вычислительную часть алгоритма от исполнительской логики игрового объекта. Игровой противник в такой структуре получает готовое решение из программного модуля и реализует соответствующее действие внутри игровой сцены.

2.3 Среда разработки и игровой движок

Unreal Engine представляет собой интегрированную среду разработки, предназначенную для создания интерактивных приложений и игровых систем. Движок объединяет инструменты построения сцен, управления объектами, обработки коллизий, анимации, визуальных эффектов и программной логики в пределах единой архитектуры.

Внутренняя архитектура Unreal Engine строится на компонентном принципе, при котором игровой объект формируется как совокупность функциональных элементов. Отдельные компоненты отвечают за отображение, столкновения, движение, взаимодействие и выполнение логики. Такой подход позволяет разделять визуальную часть объекта и его внутреннее поведение.

В рамках дипломного проекта движок используется как среда функционирования программного модуля адаптивного поведения противников. Игровая система обеспечивает двумерное пространство, обработку столкновений, переходы между комнатами и обновление состояний игровых объектов.

Для двумерной части проекта используется Paper2D, обеспечивающий работу со спрайтами, покадровой анимацией и тайловыми картами. С его

помощью формируется игровое пространство, состоящее из комнат, переходов и статических элементов уровня.

Визуальные эффекты столкновений и кратковременных игровых событий реализуются средствами Niagara, что позволяет связывать игровые события с эффектами попадания и уничтожения объектов.

Blueprints используются для вспомогательных настроек игровых объектов, тогда как основная логика программного модуля реализуется средствами C++.

Преимущества использования Unreal Engine:

- 1 Прямая интеграция с C++.
- 2 Компонентная организация игровых объектов.
- 3 Наличие Paper2D для двумерной разработки.
- 4 Поддержка визуальных эффектов через Niagara.

Использование Unreal Engine позволяет сохранить инженерный акцент на программной части, поскольку игровая среда предоставляет инфраструктуру для реализации основного объекта разработки.

2.4 Среда разработки Visual Studio

Visual Studio представляет собой интегрированную среду разработки, предназначенную для создания, сопровождения и отладки программных проектов различной сложности. Среда поддерживает C++, C#, Python и предоставляет инструменты анализа кода, отладки и профилирования.

В рамках дипломного проекта Visual Studio используется как основная среда программной реализации игровых классов и вычислительных модулей. Поскольку основная часть разработки связана с построением собственного модуля адаптивного поведения, требуется инструмент, позволяющий сопровождать исходный код и контролировать выполнение логики.

Среда обеспечивает прямую интеграцию с Unreal Engine, что позволяет автоматически подключать игровые классы к структуре проекта движка. Visual Studio используется для генерации исходных файлов классов, реализации методов и сопровождения программной архитектуры.

Одним из преимуществ является развитая система анализа кода: редактор определяет типы данных, связи между классами и доступные методы, что снижает вероятность синтаксических ошибок.

Отдельное значение имеет встроенный отладчик, позволяющий пошагово анализировать выполнение программы, значения переменных и переходы между состояниями, что особенно важно для FSM.

Преимущества использования Visual Studio:

- 1 Полная интеграция с Unreal Engine.
- 2 Удобная работа с иерархией игровых классов.
- 3 Инструменты навигации по проекту.
- 4 Поддержка Git.

Visual Studio используется как основной инструмент реализации вычислительной части проекта.

2.5 База данных SQLite

SQLite представляет собой встроенную реляционную базу данных, предназначенную для локального хранения информации внутри приложения без отдельного серверного процесса и подключения к сети. Вся структура данных сохраняется в одном локальном файле, а работа выполняется через SQL-запросы.

В рамках дипломного проекта база данных используется для хранения параметров метапрогрессии, накопленных улучшений, статистики прохождений и пользовательских настроек.

Основная игровая логика, включая обработку состояний объектов, вычисление поведения противников и столкновения, выполняется в оперативной памяти. По этой причине база данных не участвует в обработке активного игрового процесса и используется только при сохранении и загрузке.

Выбор SQLite обусловлен тем, что структура данных проекта имеет ограниченный объем и не требует серверной архитектуры.

Преимущества использования SQLite:

- 1 Отсутствие отдельного сервера.
- 2 Простая интеграция с программной частью проекта.
- 3 Низкие требования к ресурсам.
- 4 Удобное локальное хранение данных.
- 5 Поддержка стандартных SQL-запросов.

SQLite соответствует архитектуре одиночного игрового приложения, где база данных выполняет вспомогательную функцию долговременного хранения.

2.6 Система контроля версий GitHub

Git представляет собой распределенную систему контроля версий, предназначенную для фиксации изменений исходного кода и управления состоянием проекта на различных этапах разработки.

В рамках дипломного проекта Git используется для сопровождения разработки программного модуля адаптивного поведения и игровой среды. Поскольку проект включает множество взаимосвязанных файлов, требуется инструмент, позволяющий безопасно вносить изменения и возвращаться к предыдущим состояниям.

Одной из ключевых возможностей Git является работа с ветвями разработки, позволяющая изолировать изменение отдельных частей проекта без нарушения основной версии.

Дополнительное значение имеет история изменений: каждая фиксация содержит временную отметку и описание этапа разработки.

GitHub используется как удаленная платформа хранения репозитория. Платформа обеспечивает резервное хранение кода, доступ к различным версиям проекта и сохранение истории разработки.

Преимущества использования Git и GitHub:

- 1 Последовательная фиксация этапов разработки.
- 2 Возврат к предыдущим версиям.
- 3 Работа с ветвями разработки.
- 4 Удаленное хранение исходного кода.
- 5 Подтверждение авторской последовательности разработки.

Использование Git и GitHub позволяет сопровождать проект как последовательный инженерный процесс.

2.7 Блок обработки пользовательского ввода

Блок обработки пользовательского ввода предназначен для получения управляющих сигналов от пользователя и их преобразования в команды, используемые внутренними подсистемами игрового приложения. В пределах игрового цикла данный блок является начальной точкой формирования действий игрока, поскольку каждое изменение положения персонажа, запуск атаки или вызов игрового меню начинается с регистрации пользовательского воздействия.

В рамках проекта управление строится на использовании клавиатурного ввода. Перемещение игрового персонажа осуществляется через набор направлений, а действия атаки формируются через отдельные команды, определяющие момент запуска боевого взаимодействия. При обработке пользовательского ввода фиксируется не только факт нажатия, но и удержание клавиши, поскольку движение персонажа должно сохраняться до завершения действия пользователя.

Дополнительно блок выполняет фильтрацию команд, которые не должны передаваться в игровые подсистемы при определенных состояниях приложения, например при открытом меню, завершении игрового цикла или переходе между комнатами.

Функции блока обработки пользовательского ввода:

- регистрация нажатий управляющих клавиш;
- определение направления перемещения персонажа;
- передача команд запуска атаки;
- ограничение передачи команд в неактивных игровых состояниях;
- формирование управляющих параметров для последующей обработки.

Связи блока с другими функциональными компонентами:

– блок управления персонажем: управляющие сигналы движения и действий передаются для изменения положения игрового объекта и запуска игровых действий;

– блок управления игровым состоянием: отдельные команды используются для изменения режима работы приложения, включая паузу и системные переходы.

Блок обработки пользовательского ввода формирует исходный поток команд, от которого зависит изменение как игровых действий, так и системных состояний внутри программной структуры проекта.

2.8 Блок управления персонажем

Блок управления персонажем предназначен для обработки команд, поступающих из блока пользовательского ввода, и преобразования их в изменение состояния игрового объекта внутри игровой сцены. В рамках проекта данный блок отвечает за движение игрового персонажа, изменение направления, запуск атакующих действий и обновление параметров состояния игрока.

Основой работы блока является постоянное обновление координат персонажа в пределах игрового пространства. После получения управляющих сигналов определяется направление перемещения, рассчитывается новое положение игрового объекта и выполняется проверка допустимости движения с учетом границ комнаты, препятствий и статических элементов уровня.

Дополнительно блок обеспечивает управление направлением атаки. В проекте реализуются атаки в фиксированных направлениях, соответствующих ориентации персонажа. При запуске атаки фиксируется текущее направление, после чего создается соответствующее боевое действие.

Функции блока управления персонажем:

- обработка параметров движения;
- обновление положения персонажа;
- определение текущего направления ориентации;
- запуск атакующих действий;
- обновление внутренних параметров состояния игрока;
- передача игровых параметров в боевые подсистемы.

Связи блока с другими функциональными компонентами:

- блок обработки пользовательского ввода: получает управляющие команды движения и действий;
- блок боевой системы: передает параметры атаки и состояние оружия;
- блок адаптивного поведения противников: предоставляет данные о положении игрока и характере действий.

Блок управления персонажем формирует игровые параметры, которые используются в вычислении поведения противников

Блок обработки пользовательского ввода формирует исходный поток команд, от которого зависит изменение игровых действий и системных состояний проекта.

2.9 Блок генерации уровня

Блок генерации уровня предназначен для формирования структуры игрового пространства, в пределах которого осуществляется последовательное прохождение игровых эпизодов. В рамках проекта игровая среда строится по комнатному принципу, где каждая зона представляет собой отдельный участок подземелья с набором препятствий.

Основой работы блока является создание взаимосвязанных комнат, соединенных переходами в пределах одного игрового цикла. При каждом

новом запуске изменяется порядок расположения комнат и направление дальнейшего продвижения внутри уровня.

Каждая комната формируется как участок тайловой карты, содержащий напольные элементы, стены, проходы и допустимые области размещения игровых объектов. После создания структуры помещения определяется расположение входных и выходных зон.

Функции блока генерации уровня:

- формирование набора игровых комнат;
- определение взаимного расположения комнат;
- построение структуры проходов;
- размещение тайлов пола и стен;
- выделение позиций появления игровых объектов;
- контроль связности игрового маршрута.

Связь блока с другим функциональным компонентом – блоком управления игровым состоянием, получает сигналы о начале нового игрового цикла, переходе между комнатами и необходимости формирования следующего уровня.

Блок генерации уровня формирует пространственную основу игровой системы и условия функционирования остальных компонентов.

2.10 Блок боевой системы

Блок боевой системы предназначен для обработки игровых столкновений, расчета атакующих действий и изменения параметров игровых объектов в результате боевого взаимодействия. В рамках проекта данный блок обеспечивает выполнение атак игрока, регистрацию попаданий и изменение состояния игровых сущностей.

Основой работы блока является контроль взаимодействия атакующих объектов с противниками и игровым персонажем. После запуска атаки определяется зона поражения или траектория движения атакующего объекта, затем выполняется проверка пересечения с игровыми объектами.

В проекте предусмотрены два типа атакующих действий: удар ближнего действия и атака движущимся объектом. Для ближнего действия используется ограниченная зона поражения, для дальнего действия создается отдельный объект с направленным движением.

Функции блока боевой системы:

- обработка атакующих действий игрока;
- расчет зоны поражения и траектории атакующих объектов;
- проверка столкновений;
- изменение параметров здоровья;
- запуск игровых событий при попадании;
- фиксация уничтожения игровых объектов.

Связи блока с другими функциональными компонентами:

- блок управления персонажем: получает параметры атаки и направление действия;

- блок адаптивного поведения противников: передает данные о столкновениях и состоянии противников;
- блок управления игровым состоянием: сообщает о завершении боевых эпизодов и уничтожении противников.

Работа блока обеспечивает формирование боевого взаимодействия как основной части игрового цикла.

2.11 Блок адаптивного поведения противников

Блок адаптивного поведения противников является центральным программным компонентом дипломного проекта и выполняет функцию выбора текущего состояния игрового противника в зависимости от параметров игровой ситуации. В отличие от стандартных схем поведения, основанных только на расстоянии до цели или факте обнаружения игрока, данный блок использует совокупность нескольких входных параметров, которые анализируются перед каждым изменением состояния.

Внутренняя логика блока построена на конечном автомате состояний, где каждому состоянию соответствует определенный тип поведения игрового объекта. В рамках проекта используются состояния ожидания, преследования, атаки, отхода и обходного перемещения. Каждое состояние задает отдельный режим движения и взаимодействия противника внутри игровой комнаты.

Состояние ожидания используется в случаях, когда противник не имеет достаточных оснований для активного взаимодействия и сохраняет исходное положение либо выполняет ограниченное движение в пределах небольшой области. При появлении игрока в зоне контроля выполняется переход к анализу расстояния и текущих игровых параметров.

Состояние преследования активируется при обнаружении игрока на допустимой дистанции и используется для сокращения расстояния до цели. В этом режиме противник изменяет свое положение в направлении игрока, учитывая препятствия и текущую геометрию комнаты.

Состояние атаки используется при достижении допустимой дистанции взаимодействия. В этом режиме выполняется запуск атакующего действия, после чего блок повторно оценивает игровую ситуацию для определения следующего состояния.

Состояние отхода применяется при снижении уровня здоровья противника, неблагоприятном положении внутри комнаты или наличии угрозы со стороны игрока. В таком режиме противник увеличивает дистанцию до цели и изменяет траекторию движения.

Состояние обходного перемещения используется для смены позиции относительно игрока без прямого приближения или отступления. Это позволяет формировать менее предсказуемую модель движения внутри комнаты.

Поверх конечного автомата в блоке реализуется дополнительный слой оценки угрозы. Перед выбором состояния рассчитывается коэффициент угрозы, учитывающий текущие параметры игровой ситуации. В расчет

включаются расстояние до игрока, используемый тип оружия, текущее состояние здоровья противника, наличие ближайших союзных объектов, плотность заполнения комнаты и информация о последних действиях игрока.

Дополнительно используется слой памяти действий игрока. Блок фиксирует недавние действия, включая запуск дальних атак, резкое сокращение дистанции и повторяющиеся действия в пределах короткого временного интервала. Полученные данные используются при последующем выборе состояния, что позволяет изменять реакцию противника в пределах одинаковых внешних условий.

Функции блока адаптивного поведения противников:

- анализ текущих параметров игровой ситуации;
- выбор состояния конечного автомата;
- выполнение переходов между состояниями;
- учет последних действий игрока;
- формирование управляющего решения для игрового противника.

Связи блока с другими функциональными компонентами:

- блок управления персонажем: получает данные о положении игрока, направлении движения и характере текущих действий;
- блок боевой системы: получает информацию о столкновениях, уровне здоровья и результатах атак;
- блок управления игровым состоянием: передает данные о завершении активности противников, изменении состояния комнаты и переходе между игровыми этапами.

Блок адаптивного поведения отделен от исполнительной логики игрового объекта и функционирует как самостоятельный вычислительный модуль, формирующий решение о поведении противника внутри общей структуры игрового приложения.

2.12 Блок управления игровым состоянием

Блок управления игровым состоянием предназначен для координации основных этапов игрового цикла и контроля переходов между различными режимами работы приложения. В пределах архитектуры проекта данный блок объединяет информацию от игровых подсистем и определяет текущее состояние игровой сессии.

Основная задача блока заключается в фиксации начала игрового цикла, контроле прохождения комнаты, определении завершения боевого эпизода и запуске перехода к следующему участку уровня.

После уничтожения всех противников в пределах комнаты блок изменяет состояние текущего эпизода и разрешает переход к следующей зоне.

Функции блока управления игровым состоянием:

- контроль начала и завершения игрового цикла;
- фиксация завершения боевых эпизодов;
- управление переходами между комнатами;
- изменение режимов работы приложения;

- координация последовательности игровых событий;
 - передача управляющих сигналов другим подсистемам.
- Связи блока с другими функциональными компонентами:
- блок генерации уровня;
 - блок боевой системы;
 - блок адаптивного поведения противников;
 - блок пользовательского интерфейса;
 - блок сохранения и метапрогрессии;
 - блок отображения игры.

Работа данного блока обеспечивает согласованность между игровыми подсистемами и поддерживает единый порядок изменения состояний внутри всего игрового приложения.

2.13 Блок пользовательского интерфейса

Блок пользовательского интерфейса предназначен для отображения игровой информации, необходимой пользователю в процессе прохождения, и обеспечивает визуальную связь между внутренним состоянием системы и действиями игрока.

Основной интерфейсный слой активен во время прохождения и содержит информацию о текущем уровне здоровья персонажа, накопленных игровых параметрах и доступных элементах взаимодействия.

При изменении состояния персонажа блок интерфейса обновляет соответствующие визуальные элементы без задержки, чтобы игрок получал актуальную информацию о последствиях боевых столкновений.

Функции блока пользовательского интерфейса:

- отображение текущего уровня здоровья персонажа;
- вывод игровых параметров;
- отображение экранов паузы и завершения прохождения;
- вывод информации о доступных улучшениях;
- обновление интерфейсных элементов.

Связь блока с другим функциональным компонентом – блоком управления игровым состоянием, получает данные о текущем режиме приложения, завершении игрового цикла и переходах между комнатами.

Работа блока пользовательского интерфейса обеспечивает постоянное отображение состояния игровой системы и делает внутренние изменения приложения доступными для восприятия пользователем в процессе прохождения.

2.14 Блок сохранения и метапрогрессии

Блок сохранения и метапрогрессии предназначен для хранения параметров, которые должны сохраняться между отдельными игровыми циклами и использоваться при последующих прохождениях.

В рамках проекта данный блок обеспечивает долговременное

накопление результатов, включая игровые улучшения, статистику прохождений и пользовательские параметры.

Сохранение выполняется в моменты завершения прохождения, получения нового улучшения или изменения постоянных параметров персонажа.

Функции блока сохранения и метапрогрессии:

- сохранение накопленных параметров;
- загрузка данных при запуске приложения;
- хранение игровых улучшений;
- фиксация статистики прохождений;
- обновление постоянных параметров персонажа.

Связь блока с другим функциональным компонентом – блоком управления игровым состоянием, получает сигналы о завершении игрового цикла и необходимости сохранения данных.

Работа данного блока обеспечивает связь между отдельными игровыми циклами и формирует накопительный слой развития внутри общей структуры проекта.

2.15 Блок отображения игры

Блок отображения игры предназначен для формирования визуального представления игровой сцены и вывода текущего состояния игровых объектов в пределах экрана пользователя.

Основой работы блока является постоянное обновление визуального состояния сцены в соответствии с положением игровых объектов. На каждом этапе игрового цикла отображаются координаты персонажа, противников, элементы уровня и дополнительные игровые объекты.

Визуальная часть проекта построена на использовании двумерных спрайтов, размещаемых в игровой сцене в соответствии с координатами игровых сущностей.

Функции блока отображения игры:

- отображение игрового пространства и структуры комнаты;
- вывод персонажа, противников и игровых объектов;
- обновление визуального положения сущностей;
- отображение покадровой анимации;
- запуск визуальных эффектов игровых событий.

Связь блока с другим функциональным компонентом – блоком управления игровым состоянием, получает сигналы о переходе между комнатами, завершении игровых эпизодов и изменении состояния сцены.

Блок отображения игры обеспечивает визуальное представление работы всех игровых компонентов.

3 ФУНКЦИОНАЛЬНОЕ ПРОЕКТИРОВАНИЕ

Раздел функционального проектирования посвящен описанию внутренней структуры программного модуля адаптивного поведения противников и основных программных компонентов игровой системы, обеспечивающих его функционирование в рамках разрабатываемого проекта. Основное внимание в разделе уделено построению иерархии классов, реализующих конечный автомат состояний, модуль оценки угрозы и слой памяти игровых действий, влияющих на выбор поведения противника в пределах текущей игровой ситуации.

Поскольку объектом разработки является не только отдельный программный модуль принятия решений, но и игровая среда, в разделе также рассматриваются программные компоненты, через которые обеспечивается взаимодействие модуля поведения с игровым персонажем, боевой системой, обработкой состояний противников и изменением параметров игрового пространства.

Отдельно описываются свойства и методы ключевых классов, отвечающих за смену состояний противника, расчет коэффициента угрозы, анализ входных параметров и передачу управляющих решений в исполнительную игровую логику. Представленная программная структура формирует основу для реализации алгоритма адаптивного поведения в условиях динамически изменяющегося игрового окружения.

Диаграмма классов представлена на чертеже ГУИР.400201.112 РР.1.

3.1 Описание структуры классов

3.1.1 Класс `PlayerCharacter`

Класс `PlayerCharacter` является основным компонентом представления пользователя в игровой среде, инкапсулирующим логику обработки ввода, контроля пространственного перемещения и управления расширенными боевыми характеристиками. Данный класс формирует центральную точку отсчета для модуля принятия решений противников, предоставляя исчерпывающую информацию о пространственном положении, ресурсах выживаемости и активных тактических действиях пользователя.

Программная реализация глубоко интегрирована с подсистемами `Paper2D` для синхронизации двумерных анимаций с физическими хитбоксами, а также опирается на компоненты обработки коллизий `Unreal Engine`. Выступая единым информационным узлом, класс транслирует события игрового процесса в алгоритм выбора поведения враждебных существ для расчета коэффициентов угрозы.

Атрибуты класса:

- `healthComponent` – фиксирует входящий урон и отображает процент текущего здоровья;
- `staminaPool` – счетчик запаса выносливости для уклонений;
- `movementSpeed` – базовая скорость перемещения персонажа;

- `currentActionState` – идентификатор текущего состояния агрессивности;
- `spriteRenderer` – модуль визуального отображения `Paper2D`;
- `combatComponent` – логика нанесения урона и хранения активных снарядов;
- `collisionCapsule` – физическая капсула для обработки столкновений;
- `activeWeaponSlot` – указатель на экипированное оружие;
- `dashCooldownTimer` – таймер перезарядки функции рывка;
- `invulnerabilityFramesFlag` – флаг временной неуязвимости после рывка;
- `inputBuffer` – буфер преднажатых клавиш управления;
- `statusEffects` – активные модификаторы состояния в реальном времени;
- `spatialData` – координаты и вектор взгляда для противников.

Методы класса:

- `ProcessInput()` – считывает команды ввода в векторы движения;
- `UpdatePosition()` – интерполирует координаты с учетом препятствий;
- `CalculateDamageYield()` – компилирует данные об исходящем уроне;
- `ExecuteDashManeuver()` – активирует рывок и блокировку урона;
- `HandleCollisionImpact()` – рассчитывает отдачу при столкновениях;
- `SwitchActiveWeapon()` – переопределяет параметры и анимации оружия;
- `RestoreStaminaPassively()` – регенерирует выносливость вне боя.

`PlayerCharacter` обеспечивает:

- бесперебойную трансформацию механического манипулирования интерфейсами ввода в математически точные игровые действия;
- глубокую интеграцию физических барьеров коллизий со сложной системой двумерных анимаций;
- трансляцию структурированного массива телеметрии, критически важного для функционирования алгоритма выбора поведения;
- обработку комплексных многослойных механик выживаемости, включая выносливость, неуязвимость и статусное воздействие.

3.1.2 Класс `EnemyBase`

Класс `EnemyBase` отвечает за конструирование фундаментальной программной архитектуры для всех подтипов враждебных компонентов среды, инкапсулируя универсальные протоколы контроля жизненного цикла, физического позиционирования и получения внешнего урона. Класс выступает низкоуровневым стандартом, регламентирующим взаимодействие противников с ядром `Unreal Engine`, включая генерацию лута, обработку

отбрасывания и связь с системой создания уровня. Архитектурный подход объединения базовых функций устраняет необходимость дублирования методов обработки коллизий, предоставляя гибкий каркас параметров настройки дистанции возможной агрессии противников, сопротивления прерываниям и маршрутов предварительного патрулирования для любых производных классов.

Атрибуты класса:

- `maxHealthTokens` – максимальный запас прочности объекта;
- `currentHealth` – текущее здоровье, определяющее фазы;
- `baseMovementCapability` – базовая скорость с учетом эффектов;
- `physicalCollider` – коллизия для вычисления попаданий;
- `enemyTypeIdentifier` – маркер типа для системной генерации;
- `aggroRadiusSettings` – радиус первичного обнаружения игрока;
- `staggerResistance` – лимит урона до прерывания анимации;
- `moveAcceleration` – модификатор ускорения передвижения.

Методы класса:

- `InitializeBaseStats()` – считывает стартовые параметры из файлов баланса;
- `ProcessReceivedDamage()` – обрабатывает получаемый урон и резисты;
- `CheckAggroConditions()` – проверяет условия начала боя;
- `ApplyPushbackImpulse()` – применяет вектор отбрасывания при уроне;
- `TriggerStaggerState()` – переводит объект в состояние ошеломления;
- `deathAnimationDuration` – время отрисовки анимации смерти;
- `spawnPointOrigin` – стартовая координата появления.

EnemyBase реализует:

- обработку комплексной математики физического взаимодействия, включая прерывание атак, отбрасывание и градиентное ускорение;
- универсализацию связи компонентов поведения с графическими, звуковыми системами и логикой процедурной генерации лута;
- формирование безопасной и стабильной программной основы для последующего наращивания слоев искусственного принятия решений.

3.1.3 Класс AdaptiveEnemy

Класс `AdaptiveEnemy` формирует сложную надстройку над базовым функционалом противника, внедряя вычислительный центр конечного автомата состояний и адаптивный слой оценки среды. Данный объект выполняет прецизионный анализ текущей тактической обстановки, суммируя дистанции, коэффициенты видимости и остаток собственного здоровья в единый вектор параметров агрессивности. Логика класса интегрирует механизм памяти коррекции, фиксирующий исходы прошедших боевых микро-эпизодов для автоматического отказа от повторяющихся

неэффективных маневров. Компонент способен координировать действия с другими сущностями в периметре, динамически вычисляя оптимальные углы обхода и точки отступления через массив трассировочных лучей среды.

Атрибуты класса:

- stateMachineReference – указатель на активный автомат состояний;
- threatEvaluatorModule – подсистема оценки выживаемости и угрозы;

- memoryCorrectionData – база результативности маневров за сессию;
- visionRange – дальность зрительного сканирования и фокуса;
- currentContextFactors – массив факторов среды для выбора команд.

Методы класса:

- EvaluateCombatContext() – формирует пакет телеметрии боя для противников;

- TriggerStateTransition() – форсирует обрыв текущего состояния;
- UpdateMemoryCorrection() – записывает результат маневра в память;

- ExecuteCurrentBehavior() – вызывает логику текущей фазы поведения.

AdaptiveEnemy выполняет:

- внедрение комплексной оценки контекста окружающей среды для управления автоматом состояний;
- интеграцию памяти ошибок, снижающую предсказуемость действий персонажей при затяжных боях;

- непрерывную математическую компенсацию движения цели с учетом физических геометрических препятствий;

- реализацию алгоритмов тактического взаимодействия в периметре небольших групп противников.

3.1.4 Класс EnemyStateMachine

Класс EnemyStateMachine выступает изолированным контроллером управления логической конфигурацией противника, строго реализуя классическую архитектуру графа состояний. Конструкция автомата регламентирует дискретную смену состояний Idle, Chase, Attack, Retreat и Flank в соответствии с матрицей разрешенных переходов. Программный интерфейс модуля избавляет классы-оболочки от необходимости самостоятельно обрабатывать проверки условий смены алгоритмов активности, концентрируя всю математику выбора маршрута в едином изолированном узле. Помимо базового переключения, класс поддерживает сложные механизмы очереди состояний, приостановки логики и временной блокировки переключений для защиты от зацикливания физических вычислений перемещения.

Атрибуты класса:

- activeStateNode – ссылка на активную базу поведения;

- stateDictionary – реестр всех доступных сценариев;
- transitionLockTimer – таймер защиты от частой смены фаз;
- transitionMatrix – граф разрешенных переходов состояний.

Методы класса:

- InitializeStates() – создает узлы состояний в памяти;
- RequestStateChange() – пропускает запрос перехода через сетку допусков;
- ProcessActiveState() – вызывает такт активного состояния;
- EvaluateTransitionConditions() – проверяет метрики для самостоятельного перехода.

EnemyStateMachine обеспечивает:

- строгую алгоритмическую классификацию действий без риска наложения противоречивых команд физики и поворотов;
- проверку математической совместимости переходов по логическому направленному графу;
- устойчивость программной среды к микропикам (дребезгу) угрозы через механизм временных блокировок и очередей фаз;
- изоляцию низкоуровневых операций проверки состояния от непосредственного вызова механических перемещений родительского класса.

3.1.5 Класс ThreatEvaluator

Класс ThreatEvaluator отвечает за непрерывный математический анализ игровой обстановки, конвертируя разрозненные факторы среды в единый нормализованный коэффициент угрозы. Он выступает центральным вычислительным ядром адаптивного слоя, обособляя логику расчета опасности от механических контроллеров персонажа. Инкапсулированные алгоритмы используют нелинейные функции для оценки дистанции, поступающего урона и плотности вражеских существ на участке карты. Вычисленный индекс опасности напрямую интегрируется в матрицу конечного автомата, аппаратно диктуя моменты перехода противников из наступательных фаз в режим сохранения ресурсов.

Атрибуты класса:

- currentThreatScore – текущий агрегированный коэффициент опасности;
- threatDecayRate – скорость затухания угрозы со временем;
- damageWeightMultiplier – индекс агрессивности при получении урона;
- distanceWeightMultiplier – важность дистанции до угрозы;
- clusterDensityThreshold – порог скученности союзников рядом.

Методы класса:

- CalculateNormalizedThreat() – рассчитывает скалярный индекс угрозы;
- ProcessDamageSpike() – повышает угрозу при получении сильного урона;

- DecayThreatGradually() – плавно снижает коэффициент угрозы вне боя;
 - EvaluateSpatialDensity() – учитывает соседних врагов в оценке риска;
 - ComputeDistanceModifier() – переводит дистанцию в степень опасности.
- ThreatEvaluator обеспечивает:
- глубокий параметрический мониторинг динамики боевых столкновений;
 - трансформацию физических событий среды в машинно-читаемые математические индексы;
 - предоставление фундаментальных данных для переключения состояний адаптивного алгоритма;
 - снижение нагрузки на главный класс противника через делегирование сложной вычислительной логики.

3.1.6 Класс MemoryModule

Класс MemoryModule отвечает за регистрацию и последующий анализ результатов боевых сценариев, формируя накопительный банк тактического опыта в рамках отдельной сессии. Архитектура компонента опирается на кольцевые буферы записей, фиксируя комбинации пространственных координат, примененных атак и полученных негативных ответов. Аналитический аппарат модуля непрерывно сопоставляет текущую диспозицию с историческим реестром неудач, динамически понижая вероятность выбора заведомо проигрышных поведенческих паттернов. Данный механизм исключает повторяемость действий виртуальных врагов, программно заставляя их адаптировать угол подхода.

Атрибуты:

- historicalRecordsBuffer – круговой буфер записей прошедших боев;
- failureThreshold – лимит неудач до алгоритмической блокировки;
- attackSuccessRates – процент результативности атак;
- learningRateMultiplier – скорость адаптации интеллекта;
- forgottenEventsTimer – очистка памяти.

Методы класса:

- RecordActionOutcome() – записывает результат боевого действия;
- EvaluateActionViability() – возвращает математический вес успешности маневра;
- PruneOldRecords() – очищает устаревшие записи из памяти.

MemoryModule реализует:

- внедрение локального самообучения через механизм регистрации позитивных и негативных исходов;
- подавление заикливания неэффективных действий автомата состояний;

- математически обоснованную корректировку весов вероятности выбора маневров;
- динамическое формирование карты опасных зон на основе накопленного опыта.

3.1.7 Класс `CombatComponent`

Класс `CombatComponent` является унифицированным интерфейсом обработки всех боевых столкновений, инкапсулирующим механику генерации, передачи и поглощения урона. Он функционирует как посредник между алгоритмами принятия решений и физическим движком Unreal Engine, управляя процессами инициации атак, создания метательных снарядов и расчета пересечений оружия. Структурно класс независим от визуальных оболочек, оперируя исключительно числовыми множителями, векторами направлений и таймерами применения навыков.

Атрибуты класса:

- `baseDamageProfile` – базовый урон, тип и сила отдачи;
- `attackCooldownRegistry` – словарь таймеров задержки для атак;
- `hitboxGenerators` – триггеры поражения для анимаций;
- `activeProjectilesList` – пул активных летящих снарядов;
- `damageMultiplier` – динамический статусный множитель атаки.

Методы класса:

- `InitiateMeleeSweep()` – выполняет замах ближнего боя по сектору;
- `SpawnRangedProjectile()` – устанавливает и запускает снаряд;
- `ApplyDamageModifiers()` – применяет улучшения на итоговый урон;
- `TrackWindupPhase()` – обслуживает фазу замаха/подготовки атаки;
- `DispatchDamageEvent()` – публикует событие попадания логам системы.

`CombatComponent` обеспечивает:

- централизацию процессов наложения и расчета математического урона;
- трассировку пересечений боевых зон с геометрией уровня и физическими объектами;
- контроль фаз подготовки, активного поражения и восстановления после инициации атаки;
- бесшовную работу с множителями улучшений и генерацией зависимых снарядов.

3.1.8 Класс `ProjectileActor`

Класс `ProjectileActor` отвечает за автономный физический контроль метательных объектов и визуальных эффектов дистанционного поражения на сцене. Объект функционирует независимо от сущности-создателя после момента создания, подчиняясь собственной логике прямолинейного движения, регистрации столкновений и таймерам самоуничтожения. Паттерн пула объектов позволяет эффективно управлять жизненными циклами сотен

летающих стрел или заклинаний без критической нагрузки на движок сборщика мусора. Класс инкапсулирует пакет урона, системы частиц и функции реактивного изменения траектории полета.

Атрибуты класса:

- `forwardVelocity` – вектор направления полета объекта в мире;
- `speedScalar` – количественная скорость прохождения дистанции;
- `damagePayload` – структура с точными параметрами урона.

Методы класса:

- `InitializeFlightTrajectory()` – задает вектор полета в момент создания;
- `UpdateSpatialCoordinates()` – смещает объект в пространстве (полет);
- `ProcessCollisionEvent()` – перехватывает триггер попадания в цель;
- `ExecuteHomingAdjustments()` – вычисляет и выполняет самонаведение.

`ProjectileActor` реализует:

- автономную обработку логики полета дистанционных атак;
- глубокую интеграцию с коллизиями для точной передачи структуры повреждений;
- поддержку комплексных механик самонаведения, сквозного пробития и возврата.

3.1.9 Класс `RoomGenerator`

Класс `RoomGenerator` представляет собой процессор процедурной компоновки игровых сегментов, отвечающий за математическое распределение, логическое соединение и расстановку помещений на глобальной сетке. Архитектура класса базируется на алгоритмах случайного блуждания, гарантирующем отсутствие пересечений слоев стен. Модуль работает исключительно в фазе загрузки уровня, потребляя сконструированные префабы комнат и структурных стыков. Итогом работы генератора выступает детерминированный массив навигационных узлов, полностью готовый к интеграции системы спавна.

Атрибуты класса:

- `gridCoordinateMatrix` – многомерная матрица занятых ячеек этажа;
- `roomTemplatePrefabs` – коллекция базовых помещений;
- `corridorPrefabs` – набор физических переходов (коридоров).

Методы класса:

- `InitializeGridMatrix()` – резервирует память под сетку уровня;
- `ExecuteRandomWalk()` – генерирует расположение комнат на сетке;
- `ValidatePositionAvailability()` – исследует ячейки на геометрические конфликты.

`RoomGenerator` обеспечивает:

- создание масштабной архитектуры этажей на базе контролируемых вероятностей;
- интеграцию локаций благодаря алгоритмам стыковки проемов;
- фундамент для размещения агрессивных объектов и зон.

3.1.10 Класс RoomController

Класс `RoomController` выступает локальным менеджером игрового процесса в масштабах конкретного сгенерированного помещения. Этот узел делегирует задачи агрессии модулям адаптивного поведения, сохраняя контроль над пространственными правилами: активацией барьеров, планированием волн врагов и выдачей наград. Архитектура позволяет замораживать логику до физического пересечения игроком триггера входа. Это экономит процессорное время, не позволяя конечному автомату дальних врагов расходовать ресурсы движка впустую.

Атрибуты класса:

- `entranceTriggerBox` – логический триггер начала боя при входе;
- `roomStateIdentifier` – актуальный статус зачистки данной комнаты;
- `physicalBarriersList` – массив барьеров, блокирующих двери.

Методы:

- `InitializeTriggerVolume()` – настраивает фильтры входа для игрока;
- `EngageCombatPhase()` – блокирует выходы и призывает противников;
- `RegisterEnemyDeath()` – регистрирует гибель конкретного моба;
- `ProcessWaveTransitions()` – управляет задержками между волнами противников.

`RoomController` реализует:

- изоляцию боевых ивентов в рамках архитектуры отдельных помещений без пересечения логик;
- глобальную оптимизацию ресурсов за счет заморозки пустых или дальних секторов;
- алгоритм локальной прогрессии с обработкой блокировки стен и выдачей лута.

3.1.11 Класс GameStateManager

Класс `GameStateManager` функционирует как глобальный контроллер состояния игрового процесса, координируя жизненные циклы всех периферийных систем, уровней и интерфейсов. Архитектурный паттерн `Singleton` гарантирует непрерывность выполнения логики координатора независимо от загрузки конкретных комнат или визуальных меню. Модуль инстанцирует подсистемы физики, вызывает процедуры генерации уровней, агрегирует данные телеметрии от внутренних контроллеров и одобряет переход между глобальными фазами приложения: главным меню, загрузочным экраном, активным игровым процессом и экранами результатов. Выступая высшим звеном иерархии, класс обладает прямым доступом к

аппаратному пулу памяти Unreal Engine для форсированной очистки устаревших объектов.

Атрибуты класса:

- `currentGlobalPhase` – идентификатор макро-состояния приложения;
- `levelGeneratorRef` – ссылка на контроллер генерации уровня;
- `persistentPlayerState` – структура длительного сохранения прогресса;
- `masterClockTimer` – таймер текущей игровой сессии;
- `activeInterfaceController` – указатель на контроллер всех интерфейсов (UI);
- `garbageCollectionThreshold` – лимит памяти для форсированной очистки мусора;
- `inputLockCounters` – счетчики блокировки управления;
- `sessionDifficultyModifier` – скаляр уровня сложности текущего забега;
- `eventGlobalBus` – центральный диспетчер событий всей системы;
- `transitionOverlayPreload` – предзагрузка экрана визуального затемнения;
- `isSimulationPaused` – флаг остановки системного времени.

Методы:

- `InitializeCoreSystems()` – выделяет память и запускает модули;
 - `RequestLevelTransition()` – безопасно подготавливает смену карты;
 - `BroadcastTimeDilation()` – транслирует замедление или паузу игры;
 - `AggregateSessionStats()` – формирует итоговую статистику забега;
 - `HandleFatalPlayerDeath()` – перехватывает сигнал смерти пользователя;
 - `UpdateDifficultyCurve()` – корректирует агрессивность противников;
 - `EnforceGarbageCollection()` – очищает ресурсы при переходе между комнатами;
 - `RegisterPeripheralSave()` – сохраняет профиль в базу данных SQLite;
 - `LockPlayerInput()` – отключает механические команды управления.
- `GameStateManager` реализует:
- перехват и корректную обработку критических событий, таких как поражение игрока или полная победа;
 - бесперебойную оркестрацию всех изолированных подсистем генерации, сохранения и графического интерфейса;
 - интеграцию алгоритмов отслеживания времени и аппаратного контроля выделения ресурсов.

3.1.12 Класс `SaveManager`

Класс `SaveManager` отвечает за реализацию архитектуры

персистентного хранения данных, формируя надежный мост между оперативной памятью игрового приложения и жестким диском устройства. Интеграция с локальной базой данных SQLite гарантирует транзакционную безопасность пакетов прогрессии, исключая вероятность фрагментации сохранений при аварийном завершении работы движка. Модуль сериализует сложные массивы инвентаря, открытых модификаторов и статистику сессий в компактные зашифрованные строки. Логика класса разделяет хранение на слои глобальной мета-прогрессии профиля и локального состояния текущего активного забега.

Атрибуты класса:

– `databaseConnectionString` – системный путь к локальной БД SQLite;

– `activeProfileSlotIdentifier` – активный слот хранения профиля;

– `serializationBuffer` – буфер для потоковой сборки данных;

– `isWritingToDiskFlag` – блокиратор конкурентных операций записи.

Методы класса:

– `EstablishDatabaseConnection()` – соединение к БД;

– `SerializeCurrentRunState()` – конвертирует прогресс в бинарный пакет;

– `ExecuteAtomicTransaction()` – отправляет транзакцию в базу единым блоком.

`SaveManager` обеспечивает:

– надежное транзакционное сохранение показателей пользователя через интеграцию архитектуры баз данных;

– защиту внутреннего прогресса профиля от потери информации при аппаратных или программных сбоях;

– разделение структур хранения локального забега и глобальной системы постоянных улучшений.

3.1.13 Класс `UpgradeManager`

Класс `UpgradeManager` представляет собой вычислительный узел управления развитием персонажа, аккумулирующий логику глобальной прогрессии и временных усилителей сессии. Компонент функционирует как централизованная матрица баланса, из которой все остальные классы-потребители забирают итоговые множители характеристик. Математический аппарат модуля поддерживает сложные алгоритмы аддитивного и мультипликативного стапывания характеристик, гарантируя корректный расчет защиты и урона без выхода за пределы конфигурационных барьеров. Класс изолирует логику применения усилений от физических объектов среды и эффектов.

Атрибуты класса:

– `permanentUpgradesRegistry` – реестр глобально открытых улучшений;

– `sessionModifiersList` – массив временных артефактов сессии;

- `baseStatTemplates` – константы базовых характеристик субъектов;
- `currentComputedStats` – итоговые вычисленные параметры с бонусами.

Методы:

- `ApplyPermanentNode()` – разблокирует и активирует глобальный талант;
- `AddSessionArtifact()` – метод добавляет временный модификатор на один забег;
- `RecalculateAllMetrics()` – обновляет параметры с учетом множителей;
- `DiscardExpiredBufs()` – удаляет эффекты по истечении их таймеров;
- `ValidatePurchaseRequirements()` – проверяет доступность узла навыков по ресурсам.

`UpgradeManager` реализует:

- математически точную оркестровку систем комплексного развития, объединяющую постоянные и временные бонусы;
- недопущение поломки игрового баланса через жесткий контроль предельных лимитов и кривых стоимости;
- разгрузку физических классов благодаря централизованной работе функций пересчета коэффициентов;
- формирование логического фундамента для подсистем торговли предметов, артефактов и экранов глобального меню.

3.1.14 Класс `UIManager`

Класс `UIManager` выступает контроллером архитектуры пользовательских интерфейсов, обеспечивая двустороннюю коммуникацию между математикой движка и графическими виджетами. Базируясь на технологии Unreal Motion Graphics (UMG), класс создает, позиционирует и уничтожает визуальные элементы окон, не вмешиваясь в логику расчетов. Компонент реализует паттерн `Observer`, подписываясь на делегаты изменения здоровья или счетчиков патронов, что исключает дорогую по ресурсам необходимость ручного опроса переменных каждый кадр. Поддерживая слои отображения, позволяя накладывать окна паузы поверх интерфейса боя с автоматической коррекцией фокуса ввода.

Атрибуты класса:

- `activeWidgetStack` – внутренний массив активных графических панелей;
- `hudPrefabAnchor` – префаб основного дисплея, баров и компаса;
- `screenResolutionScale` – множитель масштабирования под дисплей пользователя;
- `currentFocusContext` – текущий адресат фокуса управления;
- `damageFloatersPool` – оптимизированный пул всплывающих цифр урона.

Методы:

- `InstantiateCoreHUD()` – создает корневой слой интерфейса;
- `PushWidgetToStack()` – открывает новое окно меню с перехватом ввода;
- `PopWidgetFromStack()` – закрывает окно меню и возвращает управление;
- `UpdateHealthBarDynamically()` – плавно уменьшает графическую шкалу здоровья;
- `SpawnDamageFloater()` – отображает вылетающий текст урона на экране.

UIManager обеспечивает:

- бесшовную интеграцию логических параметров среды с визуальным форматом интерфейсов через паттерны делегатов;
- управление приоритетами перекрывающихся панелей без наложения конфликтов фокуса управления вводом;
- снижение программной нагрузки на движок через работу со слепками состояний и пулами вылетающих цифр;
- предоставление гибкой основы для расширения системы инвентарей, магазинов и таблиц послематчевой статистики.

3.1.15 Класс `WeaponComponent`

Класс `WeaponComponent` инкапсулирует программную логику функционирования индивидуальных архетипов вооружения, будучи подключаемым модулем для классов-персонажей. Данный узел полностью абстрагирует методы выстрелов и перезарядки от физической оболочки владельца. Позволяя игроку изменять оружейный профиль «на лету» простым переопределением ссылок внутри компонента. Контроллер управляет созданием массивов `ProjectileActor`, генерирует векторы отдачи для системы камеры и координирует тайминги анимаций, гарантируя жесткую синхронизацию выстрелов с графическим рендером.

Атрибуты:

- `weaponArchetypeData` – базовые конфигурации разброса и дальности;
- `fireRateInterval` – лимит времени между двумя выстрелами;
- `magazineCapacity` – боезапас до фазы обязательной перезарядки;
- `currentAmmoCount` – вместимость патронов на текущий момент;
- `baseDamagePayload` – структура передаваемого пакета урона;

Методы класса:

- `AttemptFireSequence()` – оценивает возможности и совершает выстрел;
- `CalculateTrajectoryVector()` – формирует линию с учетом параметров разброса;
- `InstantiateProjectile()` – создает и толкает физическую сущность снаряда;

- `InitiateReloadCycle()` – восстанавливает обойму после задержки анимации.

`WeaponComponent` реализует:

- симуляцию паттернов ведения боя, изолированную от контроллеров передвижения;
- управление вторичными объектами, такими как системы частиц вспышек и капсулы снарядов;
- управление аудиовизуальными эффектами в ответ на действия игрока.

3.1.16 Класс `EnemySpawner`

Класс `EnemySpawner` выступает высокоуровневым контроллером генерации и маршрутизации популяции противников внутри определенных секторов уровня. Класс руководствуется кривой нарастания угрозы и конфигурациями, выдавая сущности строго порциями (волнами). Это обеспечивает плавное распределение вычислительной нагрузки Unreal Engine на физический движок.

Атрибуты класса:

- `activeSpawnPoints` – отсортированные координаты;
- `currentWaveIndex` – индекс актуальной волны спавна (агрессия);
- `waveConfigurationList` – профиль числа и типа создаваемых противников;
- `playerDistanceThreshold` – радиус безопасной зоны вокруг игрока.

Методы:

- `EvaluateSpawnSafety()` – отфильтровывает точки в прямом обзоре камеры;
- `InitializeNextWave()` – калибрует сложность и количество врагов волны;
- `InstantiateEnemyEntity()` – формирует физическую модель противника;
- `ProcessDeathDelegates()` – отслеживает и фиксирует смерти вызванных противников;
- `DisplaySpawnWarning()` – рисует индикатор перед прямым появлением угрозы.

`EnemySpawner` обеспечивает:

- плавное нарастание уровня сложности посредством механизмов регулируемых временных волн.

3.1.17 Класс `NavigationHelper`

Класс `NavigationHelper` отвечает за математическую обработку пространственных данных возможных и доступных перемещений противников в рамках двумерной сетки комнаты уровня. Вычислительный аппарат узла делегирует ресурсоемкие задачи построения маршрутов асинхронным потокам движка. Узел непрерывно анализирует динамические препятствия генерируя корректирующие векторы для обхода различных

препятствий в виде других противников и объектов. Модуль транслирует нормализованные координаты напрямую в систему управления скоростью персонажей снижая задержки при смене направления. Данный подход гарантирует стабильное следование адаптивных агентов за целью в условиях постоянно меняющейся геометрии комнат.

Атрибуты класса:

- navigationMeshReference – ссылка на сетку проходимости (navmesh);
- activePathPoints – узлы транзитного маршрута на текущий момент;
- pathfindingTimeout – задержка перед новым вызовом алгоритма поиска пути;
- obstacleAvoidanceRadius – радиус математического обхода мелких преград;
- dynamicBlockersList – реестр временных стен или барьеров.

Методы класса:

- CalculateOptimalPath() – строит векторный кратчайший путь до точки;
- ValidateCurrentRoute() – проверяет маршрут на появление новых помех;
- SmoothTrajectoryCorners() – сглаживает геометрию пути при резких поворотах.

NavigationHelper обеспечивает:

- интеграцию системы динамического перестроения путей при изменениях статуса разрушаемого окружения;
- снижение процессорной нагрузки посредством кэширования лучей и сглаживания геометрических отклонений;
- фундамент контроля пространственного позиционирования для переходов автомата состояний.

3.1.18 Класс MetaProgressionData

Класс MetaProgressionData представляет собой защищенную структуру глобального профиля аккумулирующую фундаментальные математические показатели развития пользователя. Модуль функционирует независимо от сессионных контроллеров сохраняя данные между перезапусками программы. Архитектура класса изолирует реестры открытых улучшений счетчики накопленной валюты и логику стоимости талантов от физических объектов. Объект применяется подсистемами сохранения для последующей записи в таблицы баз данных. Расчеты модификаторов базовых характеристик выполняются исключительно на основе констант роста.

Атрибуты класса:

- globalExperiencePool – счетчик общего пула накопленного опыта;
- unlockedNodesRegistry – словарь приобретенных в профиле талантов;
- talentRefundTokens – счетчик доступных сбросов распределенных

талантов;

- `permanentDamageBonus` – накопленный коэффициент глобального усиления исходящего урона;

- `unlockedDifficultyTiers` – реестр открытых уровней сложности;

- `baseStatUpgrades` – массив глобальных апгрейдов характеристик;

- `maximumHealthLevel` – индекс увеличения показателя максимального здоровья;

- `currencyMultiplierLevel` – процент бонуса к добыче валюты и лута.

Методы класса:

- `ValidateUpgradePurchase()` – проводит транзакцию покупки глобального навыка;

- `CompileGlobalModifiers()` – компилирует все таланты в единые множители;

- `AddExperiencePoints()` – прибавляет чистую экспу по окончании сессии.

`MetaProgressionData` обеспечивает:

- надежное структурирование параметров профиля изолированное от логики физического взаимодействия;

- алгоритмически выверенную прогрессию стоимости разблокируемых узлов;

- безопасное накопление глобального опыта пользователя минуя риски переполнения или случайного сброса.

3.1.19 Класс `SQLiteManager`

Класс `SQLiteManager` выполняет функции специализированного драйвера управления транзакциями между оперативной памятью приложения и локальным физическим файлом базы данных. Модуль инкапсулирует синтаксис SQL-запросов выполняя процедуры атомарной записи и извлечения строк без блокировки основного потока. Интеграция с СУБД `SQLite` защищает целостность профилей при аппаратных перезагрузках или сбоях генерации. Использование пула подключений минимизирует время выполнения дисковых операций ввода-вывода.

Атрибуты класса:

- `databaseConnectionRef` – дескриптор соединения с физической базой;

- `transactionQueueBuffer` – программный накопитель коммитов для базы;

- `isQueryExecutingFlag` – индикатор занятости асинхронного потока `SQLite`;

- `sqlStatementCache` – индекс команд оптимизации.

Методы класса:

- `EstablishSecureConnection()` – инициализирует запрос соединения к БД;

- `ExecutePreparedQuery()` – отправляет конкретную SQL-команду в

движок;

- `ConstructDatabaseSchema()` – проверяет и создает таблицы при обновлении;

- `CommitTransactionBatch()` – фиксирует изменения в постоянной памяти;

- `RollbackFailedTransaction()` – откатывает состояние БД при сбое.

`SQLiteManager` обеспечивает:

- бесперебойную архитектурную связку приложения Unreal Engine с надежным форматом реляционного хранения;

- выполнение атомарных транзакций исключающих фрагментацию прогресса при критических сбоях;

- интеграцию протоколов проверки целостности и шифрования для защиты сохранений от модификации.

3.1.20 Класс `VisualEffectController`

Класс `VisualEffectController` является выделенным диспетчером графических модулей ответственным за синхронизацию и уничтожение систем частиц Niagara. Архитектура контроллера централизует запросы физических классов на отрисовку эффектов снимая с них задачи манипулирования эмиттерами. Модуль работает с пулами готовых графических объектов переиспользуя их для отрисовки следов взмахов попаданий и других эффектов. Управление жизненным циклом частиц гарантирует отсутствие утечек видеопамати при массовых сражениях на процедурных уровнях.

Атрибуты класса:

- `particleSystemsPool` – реестр загруженных элементов систем частиц;

- `activeEmittersList` – перечень отслеживаемых запущенных эффектов;

- `cullingDistanceLimits` – дистанция, за которой отключается рендер спецэффектов.

Методы класса:

- `ReserveEmitterFromPool()` – запрашивает систему частиц из пула памяти;

- `AttachEffectToSocket()` – жестко прикрепляет эффект к движущемуся сокету;

- `ReleaseEffectResources()` – обнуляет таймеры эффекта;

- `ApplyMaterialOverride()` – включает динамический шейдер статуса на сетке.

`VisualEffectController` обеспечивает:

- жесткий контроль за потреблением ресурсов видеокарты посредством обрезки графики вне поля зрения;

- независимое обслуживание систем частиц изолируя базовую физическую логику классов от рендеринга;

- адаптивное масштабирование визуальных эффектов сохраняющее производительность сцен с высокой плотностью объектов;
- бесшовную накладку динамических материалов и текстур на физические оболочки участников боя.

3.1.21 Класс EnemyPerceptionComponent

Класс `EnemyPerceptionComponent` выполняет функции сенсорного аппарата реализую алгоритмы обнаружения цели в заданном пространстве. Компонент абстрагирует математику трассировки лучей и расчеты конусов видимости изолируя процессорные затраты от центрального автомата состояний. Модуль симулирует периферическое зрение слух и тактильное восприятие противника переводя геометрические данные о дистанции в скалярные коэффициенты угрозы. Объект непрерывно обновляет внутренний реестр раздражителей передавая готовую оценку ситуации контроллерам поведения.

Атрибуты класса:

- `visualSightRadius` – дальность сканирования пространства;
- `peripheralVisionAngle` – угол математического ограничения периферийного зрения.

Методы класса:

- `PerformConeSweep()` – выполняет трассировку в пределах сектора обзора;
- `TraceAcousticEvent()` – фиксирует шум и обращает внимание противника;
- `EvaluateStimulusPriority()` – сравнивает раздражители и выбирает главную цель.

`EnemyPerceptionComponent` обеспечивает:

- бесперебойную симуляцию сенсорного восприятия с учетом визуальных препятствий;
- фундамент для активации состояний преследования и адаптивного поиска при потере контакта.

3.1.22 Класс StateTransitionController

Класс `StateTransitionController` отвечает за жесткую регламентацию переходов между фазами поведения внутри конечного автомата пресекая конфликтующие программные состояния. Модуль анализирует входящие запросы от сенсорных компонентов маршрутизаторов и счетчиков здоровья одобряя или отклоняя смену текущего алгоритма. Архитектура класса позволяет динамически подменять правила перехода на основе уровня сложности или элитного статуса противника развивая адаптивность системы. Переходы между агрессией и защитой сопровождаются автоматическим сбросом устаревших переменных и очисткой памяти траекторий.

Атрибуты класса:

- `stateMatrixDefinition` – жесткая матрица разрешенных логических переходов;
- `currentActiveStateEnum` – идентификатор выполняемой фазы;
- `transitionCooldownTimer` – задержка на частую смену алгоритма;
- `interruptedActionFlag` – флаг экстренного прерывания атакующей анимации;
- `priorityOverrideQueue` – очередь приоритетных системных скриптов.

Методы класса:

- `RequestStateChange()` – пропускает запрос перехода через сетку допусков.

`StateTransitionController` обеспечивает:

- надежную защиту конечного автомата от программных ошибок вызываемых одновременными логическими запросами;
- адаптивную модификацию графа поведений на основе анализа степени угрозы и типа противника.

3.1.23 Класс `RunStatisticsManager`

Класс `RunStatisticsManager` является центральным хранилищем аналитических метрик функционирующим исключительно в рамках отдельного старта. Модуль осуществляет сбор структуризацию и глубокий математический анализ действий пользователя формируя итоговую оценку его эффективности. Компонент перехватывает системные делегаты нанесения урона поглощения лечения и времени зачистки комнат не вмешиваясь в физику процесса. Пакеты накапливаемых данных применяются алгоритмами `GameStateManager` для динамической адаптации сложности и регуляции кривой препятствий. При завершении сеанса объект выполняет компрессию собранных чисел передавая итоговый отчет менеджеру глобальных сохранений.

Атрибуты класса:

- `totalDamageDealt` – абсолютный счетчик нанесенного за попытку урона;
- `damageReceivedCounter` – сумма полученных игроком очков повреждений;
- `roomsClearedTally` – количество зачищенных помещений в этой сессии.

Методы класса:

- `RecordDamageEvent()` – сортирует и записывает факт обмена уроном;
- `InitializeRoomTimer()` – отслеживает время зачистки локации;
- `IncrementEnemyKill()` – заносит класс поверженного объекта в статистику.

`RunStatisticsManager` обеспечивает:

- регистрацию каждого действия пользователя;
- формирование аналитической базы для работы систем адаптивной

подстройки сложности движка;

– подготовку данных для визуализации итогового прогресса.

3.1.24 Класс UpgradeSelectionManager

Класс UpgradeSelectionManager выполняет алгоритмическую выборку наград формируя структурированные пулы лотереи при завершении арен. Архитектура модуля опирается на системы псевдослучайной генерации с интегрированными механизмами защиты от невезения гарантируя математическую справедливость распределения. Компонент анализирует текущий глобальный инвентарь отсеивая заблокированные константы и выделяя синергирующие элементы. Узел напрямую коммуницирует с интерфейсом UIManager отправляя готовые пакеты данных для отрисовки графических панелей выбора. Динамическое изменение весов вероятности позволяет алгоритму адаптироваться под стиль пользователя подкидывая наиболее востребованные улучшения.

Атрибуты класса:

– randomNumberGeneratorSeed – основа генерации;
– availableUpgradesPool – словарный фильтр доступных для выпадения наград;
– rarityWeightModifiers – массив с шансами на выбивание редких артефактов.

Методы класса:

– InitializeDropPool() – считывает из профиля только доступные к выпадению предметы;
– ExecuteSelectionRoll() – создает перечень предложений на основе весов;
– AdjustWeightsForSynergy() – завышает вероятность получения компонентов синергии.

UpgradeSelectionManager обеспечивает:

– процедурную генерацию предложений наград;
– адаптируемый механизм подстройки пула выдачи под сформированный стиль и синергии конкретного забега.

3.1.25 Класс EnemyStateController

Класс EnemyStateController является центральным программным узлом модуля принятия решений объединяющим функционал конечного автомата состояний и адаптивного аналитического слоя. Компонент осуществляет непрерывный мониторинг окружающей среды пропуская сенсорные данные через математические фильтры оценки контекста. Метод изолирует базовую логику перемещения от высокоуровневых директив перехватывая управление персонажем. Модуль гарантирует плавный переход между дискретными фазами бездействия и активной агрессии исключая резкие разрывы анимаций. Заложенные алгоритмы выбора поведения динамически корректируют тактику противников опираясь на вычисленный

коэффициент угрозы и исправляя свои паттерны на основе матриц памяти.

Атрибуты класса:

- `activeBehaviorPhase` – системный идентификатор текущего маневра;
- `threatEvaluationModule` – встроенный компонент математической оценки риска;
- `memoryCorrectionCache` – журнал неудачных действий для их избегания;
- `contextUpdateInterval` – таймер перепроверки ситуации на арене;
- `healthThresholdTriggers` – границы потери очков здоровья, меняющие манеру боя;
- `navigationInterfaceRef` – ссылка на навигатор обхода препятствий;
- `targetLineOfSightStatus` – статус наличия зрительного контакта с противником;
- `engagementDistanceLimits` – зоны ближнего и дальнего боя;
- `recentActionsLog` – защита от повторений одних маневров.

Методы класса:

- `InitializeFiniteStateMachine()` – инициализирует графы и алгоритмы патруля;
- `TransitionToChaseState()` – включает механизм погони до цели;
- `ExecuteFlankManeuver()` – рассчитывает и обходит игрока с боковых позиций;
- `EvaluateRetreatNecessity()` – заставляет моба отступить при пиковой угрозе;
- `EnterAttackSequence()` – блокирует перемещение для выполнения удара;
- `ProcessMemoryCorrection()` – пересчитывает приоритеты по итогам прошлых неудач;
- `RestToIdleCondition()` – переводит логику в пассивное восстановление;
- `RecalculateThreatCoefficient()` – рассчитывает коэффициент угрозы;
- `InterruptCurrentBehavior()` – прерывает логику при ошеломлении;
- `SynchronizeAdaptiveLayer()` – обновляет алгоритмы на основе адаптивного контекста.

`EnemyStateController` обеспечивает:

- оркестрацию дискретных состояний от пассивного наблюдения и укрытия до активного маневрирования;
- интеграцию адаптивных математических алгоритмов в структуру автоматов состояний;
- подстройку поведения группы противников опирающуюся на коэффициенты угрозы.

7 ТЕХНИКО-ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ ПРОГРАММНОГО МОДУЛЯ АДАПТИВНОГО ПОВЕДЕНИЯ ПЕРСОНАЖЕЙ 2D-ИГРЫ В ЖАНРЕ ACTIONRPG

7.1 Характеристика программного средства, разрабатываемого по индивидуальному заказу

Целью разработки является создание программного модуля адаптивного поведения противников для двумерной игры жанра ActionRPG, предназначенного для повышения вариативности игровых столкновений и снижения предсказуемости поведения игровых объектов в условиях повторяющегося игрового цикла.

Разрабатываемое программное средство ориентировано на применение в составе игровых проектов, где требуется формирование изменяемой модели поведения противников без постоянного увеличения количества игровых сущностей и без расширения числа базовых игровых механик. Практическая задача разработки связана с тем, что в игровых системах данного жанра при многократном повторении игровых ситуаций устойчивые шаблоны поведения противников быстро становятся узнаваемыми, что снижает различие между отдельными игровыми эпизодами.

Программное средство решает задачи выбора поведения игровых противников в зависимости от параметров текущей игровой ситуации, включая расстояние до игрока, текущее состояние противника, плотность игрового пространства и последовательность действий игрока. Дополнительно игровая часть проекта выполняет функцию среды функционирования разработанного программного модуля и обеспечивает проверку его работы в условиях перемещения между комнатами, боевых взаимодействий и повторных игровых циклов.

Разработка программного средства рассматривается как работа проектной группы, включающей программиста игровой логики, технического дизайнера игрового контента и специалиста по тестированию и интеграции. Программист игровой логики выполняет построение программного модуля поведения, реализацию вычислительных алгоритмов и внутреннюю архитектуру проекта. Технический дизайнер обеспечивает подготовку игровой среды, настройку игровых объектов, организацию уровней и интеграцию визуальных ресурсов. Специалист по тестированию и интеграции выполняет проверку корректности игровых сценариев, устойчивости переходов между состояниями и согласованности взаимодействия программных блоков.

Необходимость разработки обусловлена несколькими факторами. В игровых проектах жанра ActionRPG устойчивый интерес пользователей поддерживается за счет повторного прохождения, изменения структуры уровней и постепенного усложнения игровых ситуаций. При этом в значительной части существующих решений поведение противников внутри

отдельных типов остается ограниченным фиксированными схемами, что приводит к повторяемости локальных столкновений даже при изменении внешних условий.

Использование разработанного программного средства позволяет изменять характер игровых столкновений без существенного увеличения объема игровых ресурсов. За счет выделения отдельного программного модуля появляется возможность расширения логики поведения противников без переработки базовой игровой архитектуры.

Практическая ценность разработки заключается в том, что программный модуль проектируется как самостоятельный компонент, который может использоваться в игровых приложениях различного типа без жесткой зависимости от конкретной структуры уровня или фиксированного набора игровых механик. Выделение алгоритма поведения в отдельный программный блок упрощает его перенос, настройку и адаптацию при использовании в других игровых проектах, где требуется изменяемая модель поведения игровых противников.

7.2 Расчет инвестиций в разработку программного средства

7.2.1 Расчет зарплат на основную заработную плату разработчиков

Для расчета затрат на основную заработную плату определяется состав исполнителей, участвующих в разработке программного средства.

В разработке программного средства принимают участие инженер-программист игровой логики, технический дизайнер игрового контента и инженер-тестировщик.

Инженер-программист игровой логики выполняет проектирование программного модуля адаптивного поведения противников, реализацию конечного автомата состояний, настройку взаимодействия игровых блоков и интеграцию программной части в игровую среду.

Технический дизайнер игрового контента отвечает за подготовку игровых ресурсов, настройку игровых сцен, организацию структуры комнат и размещение игровых объектов внутри проекта.

Инженер-тестировщик выполняет проверку корректности игровых сценариев, контроль переходов между состояниями противников и выявление отклонений в работе программного модуля.

Затраты на основную заработную плату рассчитаны по формуле [19]:

$$Z_o = K_{\text{пр}} \sum_{i=1}^n Z_{\text{чи}} \cdot t_i, \quad (7.1)$$

где $K_{\text{пр}}$ – коэффициент премий и иных стимулирующих выплат;

n – категории исполнителей, занятых разработкой программного средства;

$Z_{\text{чи}}$ – часовая заработная плата исполнителя i -й категории, р;

t_i – трудоемкость работ, выполняемых исполнителем i -й категории, ч.

Средний месячный оклад инженера-программиста игровой логики принимается на уровне 4860 рублей, технического дизайнера игрового контента – 3125 рублей, инженера-тестировщика – 2870 рублей.

Основной объем разработки программной части занимает 160 часов работы инженера-программиста игровой логики. Подготовка игровых ресурсов требует 76 часов работы технического дизайнера игрового контента. Проверка игровых сценариев и контроль устойчивости системы требуют 80 часа работы инженера-тестировщика.

Затраты на основную заработную плату приведены в таблице 7.1.

Таблица 7.1 – Затраты на основную заработную плату

Категория исполнителя	Месячный оклад, р	Часовой оклад, р	Трудоемкость работ, ч	Итого, р
Инженер-программист игровой логики	4 860,00	29,25	160	4 680,00
Технический дизайнер игрового контента	3125,00	19,53	76	1484,30
Инженер-тестировщик	2870,00	17,94	80	1507,00
Итого				7 671,30
Премия и иные стимулирующие выплаты (0%)				0
Всего затраты на основную заработную плату разработчиков				7 671,30

7.2.2 Расчет затрат на дополнительную заработную плату разработчиков

В состав дополнительной заработной платы включаются выплаты, связанные с оплатой отпусков, компенсациями за неотработанное время и иными обязательными начислениями, предусмотренными в составе фонда оплаты труда.

Расчет затрат на дополнительную заработную плату команды разработчиков выполняется по формуле [19]:

$$З_{д} = \frac{З_{о} \cdot Н_{д}}{100}, \quad (7.2)$$

где $Н_{д}$ – норматив дополнительной заработной платы.

Норматив дополнительной заработной платы принимается в размере 10 % от основной заработной платы разработчиков.

Размер дополнительной заработной платы составляет 767,13 рубля..

7.2.3 Расчет отчислений на социальные нужды

Размер отчислений на социальные нужды определяется в соответствии с установленным нормативом обязательных страховых отчислений.

В расчете используется ставка 35%, актуальная на март 2026 года, из которых 29% отчисляется на пенсионное страхование и 6% выделяется на социальное страхование.

Расчет отчислений на социальные нужды выполняется по формуле [19]:

$$P_{\text{соц}} = \frac{(3_o + 3_d) \cdot H_{\text{соц}}}{100}, \quad (7.3)$$

где $H_{\text{соц}}$ – норматив отчислений в ФСЗН.

Размер отчислений на социальные нужды составляет 2953,45 рубля.

7.2.4 Расчет прочих расходов

Прочие расходы включают затраты, связанные с организацией рабочего места, использованием вычислительной техники, электроэнергией, программными средствами и сопровождением процесса разработки.

Расчет выполняется по нормативу прочих расходов, принимаемому в размере 30 % от основной заработной платы разработчиков.

Расчет прочих расходов выполняется по формуле [19]:

$$P_{\text{пр}} = \frac{3_o \cdot H_{\text{пр}}}{100}, \quad (7.4)$$

где $H_{\text{пр}}$ – норматив прочих расходов.

Размер прочих расходов составляет 2301,39 рубля.

7.2.5 Расчет плановой прибыли, включаемой в цену программного средства

Плановая прибыль определяется исходя из полной суммы затрат на разработку программного средства и принятого уровня рентабельности.

Рентабельность затрат на разработку принимается в размере 22%. Плановая прибыль вычисляется по формуле [19]:

$$P_{\text{п.с}} = \frac{3_p \cdot P_{\text{п.с}}}{100}, \quad (7.5)$$

где $P_{\text{п.с}}$ – рентабельность затрат на разработку программного средства;

3_p – затраты на разработку программного средства.

Размер плановой прибыли составляет 3012,52 рубля.

7.2.6 Расчет затрат на разработку и цена программного средства

Общая сумма затрат на разработку определяется как сумма основной заработной платы разработчиков, дополнительной заработной платы, отчислений на социальные нужды и прочих расходов. Расчет выполняется по

формуле [19]:

$$З_p = З_o + З_d + P_{\text{соц}} + P_{\text{пр}}, \quad (7.6)$$

Цена программного средства определяется как сумма общих затрат на разработку и плановой прибыли.

Вычисление происходит по следующей формуле [19]:

$$Ц_{\text{п.с}} = З_p + П_{\text{п.с}}, \quad (7.7)$$

Пошаговое формирование цены программного средства на основе затрат приведено в таблице 7.2.

Таблица 7.2 – Затраты на разработку

Название статьи затрат	Формула/таблица для расчета	Значение, р.
1 Основная заработная плата разработчиков	Таблица 7.1	7 671,30
2 Дополнительная заработная плата разработчиков	$З_d = \frac{7\,691,30 \cdot 10}{100}$	767,13
3 Отчисление на социальные нужды	$P_{\text{соц}} = \frac{(7\,673,30 + 767,13) \cdot 35}{100}$	2 953,45
4 Прочие расходы	$P_{\text{пр}} = \frac{7\,671,20 \cdot 30}{100}$	2 301,39
5 Общая сумма затрат на разработку и реализацию	$З_p = 7\,671,30 + 767,13 + 2\,953,45 + 2\,301,39$	13693,27
6 Плановая прибыль, включаемая в цену программного средства	$П_{\text{п.с}} = \frac{13693,27 \cdot 22}{100}$	3012,52
7 Отпускная цена программного средства	$Ц_{\text{п.с}} = 13693,27 + 3012,52$	16 705,79

7.3 Расчет разработки и использования программного средства

Для организации-разработчика экономическим результатом является чистая прибыль, полученная после реализации программного средства.

Учитывая, что программное средство реализовано по отпускной цене, которая сформирована на основе затрат на разработку организацией-разработчиком, то прирост рассчитывается следующим образом:

$$\Delta\Pi_{\text{ч}} = \Pi_{\text{п.с}} \cdot \left(1 - \frac{H_{\text{п}}}{100}\right), \quad (7.8)$$

где $H_{\text{п}}$ – ставка налога на прибыль, %.

На март 2026 года ставка налога на прибыль составляет 20%.

Вычисление чистой прибыли производится по формуле (7.8):

$$\Delta\Pi_{\text{ч}} = 3012,52 \cdot \left(1 - \frac{20}{100}\right) = 2410,02 \text{ р.}$$

7.4 Расчет показателей экономической эффективности разработки программного средства

Для оценки экономической эффективности разработки определяется показатель рентабельности инвестиций в разработку программного средства. Расчет выполняется по формуле:

$$P_{\text{и}} = \frac{\Delta\Pi_{\text{ч}}}{Z_{\text{р}}} \cdot 100\%, \quad (7.9)$$

где $\Delta\Pi_{\text{ч}}$ – прирост чистой прибыли, полученной от разработки программного средства организацией-заказчиком;

$Z_{\text{р}}$ – затраты на разработку программного средства организацией-разработчиком.

Вычисление рентабельности инвестиций по формуле (7.9):

$$P_{\text{и}} = \frac{2410,02}{13693,27} \cdot 100\% = 17,6\%.$$

7.5 Вывод об экономической целесообразности реализации проектного решения

Проведенные расчеты позволяют сделать вывод о целесообразности разработки программного средства с точки зрения затрат на создание и ожидаемого экономического результата. Полученные показатели показывают, что разработка программного модуля и игровой среды сопровождается приемлемым уровнем затрат и обеспечивает положительный экономический результат при условии реализации как законченного программного решения.

Общая сумма затрат на разработку составила 13693,27 белорусского рубля. Размер плановой прибыли, включаемой в цену программного средства, составил 3012,52 белорусского рубля. После учета налога на прибыль величина чистой прибыли составляет 2410,02 белорусского рубля. Расчет рентабельности инвестиций показал значение 17,6 %.

Полученный уровень рентабельности соответствует допустимым

значениям для разработки специализированного программного средства, выполняемого по индивидуальному проектному заданию. Для программных разработок, ориентированных на ограниченный объем внедрения и отсутствие массового тиражирования на начальном этапе, подобный показатель рассматривается как достаточный для подтверждения экономической оправданности разработки.

Дополнительное значение имеет структура самого программного решения. Разрабатываемый программный модуль адаптивного поведения противников создается как самостоятельный компонент, который может использоваться отдельно от конкретной игровой среды. Это позволяет применять разработанный алгоритм в игровых проектах различной структуры без полной переработки программной логики.

Экономическая эффективность также определяется тем, что дальнейшее использование программного модуля требует меньших затрат по сравнению с первичной разработкой. При повторной адаптации в составе других игровых приложений сохраняется основная программная архитектура, а изменения затрагивают только параметры интеграции и отдельные игровые условия.

Снижение затрат при повторном использовании достигается за счет того, что вычислительная часть, отвечающая за выбор поведения игровых противников, отделена от конкретной реализации игровых объектов и не требует повторного проектирования базовых алгоритмов состояний. Это создает предпосылки для использования разработанного решения в учебных, демонстрационных и прикладных игровых проектах, где требуется изменяемая модель поведения без разработки нового программного ядра.

Отдельным фактором экономической целесообразности является ограниченный объем используемых технических ресурсов при разработке. В проекте применяются стандартные программные средства, не требующие приобретения специализированного оборудования или организации отдельной вычислительной инфраструктуры, что снижает совокупные затраты на создание и сопровождение программного решения.

Полученный программный результат сочетает самостоятельный вычислительный модуль и игровую среду для его функционирования, что расширяет возможности последующего использования и доработки. На основании выполненных расчетов разработка программного средства может рассматриваться как экономически оправданная, поскольку полученные показатели затрат, прибыли и рентабельности подтверждают целесообразность реализации выбранного проектного решения.

ЗАКЛЮЧЕНИЕ

В ходе практики была сформирована общая структура разрабатываемого программного модуля и определены основные направления дальнейшей доработки и реализации. Выполненная работа охватывает этапы проектирования, начиная с анализа предметной области и заканчивая экономическим обоснованием проектного решения.

В процессе выполнения работы были определены цели и задачи проекта, обоснована актуальность выбранной темы и рассмотрены особенности предметной области. Анализ существующих игровых решений позволил определить сильные и слабые стороны аналогов, а также выделить подходы, применяемые в современных проектах.

В рамках системного проектирования была сформирована архитектура программной системы, включающая основные функциональные блоки игрового приложения: обработку пользовательского ввода, управление персонажем, генерацию уровня, боевую систему, блок адаптивного поведения противников, управление игровым состоянием, пользовательский интерфейс, сохранение данных и отображение игры. Для каждого блока определены его функции и связи с другими блоками.

Функциональное проектирование позволило определить внутреннюю организацию программного модуля адаптивного поведения, основанного на конечном автомате состояний и дополнительном слое оценки угрозы. Были выделены основные параметры, участвующие в выборе состояния противника, включая расстояние до игрока, состояние здоровья, плотность игрового пространства и информацию о последних действиях игрока. Это позволило сформировать основу самостоятельного вычислительного механизма, отделённого от исполнительской логики игровых объектов.

Определены язык программирования C++, игровой движок Unreal Engine, среда разработки Visual Studio, база данных SQLite и система контроля версий GitHub.

В экономическом разделе выполнено технико-экономическое обоснование разработки, включающее расчет затрат на создание программного модуля и среды функционирования, определение плановой прибыли и оценку экономической эффективности проектного решения. Полученные результаты показывают, что разработка обладает практической целесообразностью и может рассматриваться как программное решение, пригодное для дальнейшего развития и адаптации под различные игровые проекты.

Выполненная работа позволила сформировать значительную часть дипломного проекта, определить дальнейшие направления его доработки и подготовить основу для завершения оставшихся разделов дипломной работы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] The Binding of Isaac [Электронный ресурс]. – Режим доступа: https://store.steampowered.com/app/250900/The_Binding_of_Isaac_Rebirth/ – Дата доступа: 17.03.2026
- [2] Enter the Gungeon [Электронный ресурс]. – Режим доступа: https://store.steampowered.com/app/311690/Enter_the_Gungeon/ – Дата доступа: 17.03.2026
- [3] Hades [Электронный ресурс]. – Режим доступа: <https://store.steampowered.com/app/1145360/Hades/> – Дата доступа: 17.03.2026
- [4] Dead Cells [Электронный ресурс]. – Режим доступа: https://store.steampowered.com/app/588650/Dead_Cells/ – Дата доступа: 17.03.2026
- [5] Moonlighter [Электронный ресурс]. – Режим доступа: <https://store.steampowered.com/app/606150/Moonlighter/> – Дата доступа: 17.03.2026
- [6] C++ [Электронный ресурс]. – Режим доступа: <https://metanit.com/cpp/tutorial/1.1.php> – Дата доступа: 17.03.2026
- [7] C# [Электронный ресурс]. – Режим доступа: <https://metanit.com/sharp/tutorial/1.1.php> – Дата доступа: 17.03.2026
- [8] Java [Электронный ресурс]. – Режим доступа: <https://metanit.com/java/tutorial/1.1.php> – Дата доступа: 17.03.2026
- [9] Unreal Engine [Электронный ресурс]. – Режим доступа: <https://habr.com/ru/articles/820477/> – Дата доступа: 17.03.2026
- [10] Unity [Электронный ресурс]. – Режим доступа: <https://habr.com/ru/articles/828302/> – Дата доступа: 17.03.2026
- [11] Godot [Электронный ресурс]. – Режим доступа: https://docs.godotengine.org/ru/4.x/getting_started/ – Дата доступа: 17.03.2026
- [12] Visual Studio [Электронный ресурс]. – Режим доступа: <https://visualstudio.microsoft.com/ru/> – Дата доступа: 17.03.2026
- [13] Git [Электронный ресурс]. – Режим доступа: <https://habr.com/ru/articles/541258/> – Дата доступа: 17.03.2026
- [14] GitHub [Электронный ресурс]. – Режим доступа: <https://docs.github.com/ru> – Дата доступа: 17.03.2026
- [15] GitLab [Электронный ресурс]. – Режим доступа: <https://docs.gitlab.com> – Дата доступа: 17.03.2026
- [16] SQLite [Электронный ресурс]. – Режим доступа: <https://metanit.com/sql/sqlite/1.1.php> – Дата доступа: 17.03.2026
- [17] PostgreSQL [Электронный ресурс]. – Режим доступа: <https://postgrespro.ru/docs/postgresql> – Дата доступа: 17.03.2026
- [18] MongoDB [Электронный ресурс]. – Режим доступа: <https://metanit.com/nosql/mongodb/1.1.php> – Дата доступа: 17.03.2026
- [19] Экономика проектных решений: методические указания по экономическому обоснованию дипломных проектов: учеб.-метод пособие / В.Г. Горовой [и др.] – Минск: БГУИР, 2021 – 107 с.

ПРИЛОЖЕНИЕ А
(обязательное)

Вводный плакат. Плакат

ПРИЛОЖЕНИЕ Б
(обязательное)

Схема структурная

ПРИЛОЖЕНИЕ В
(обязательное)

Диаграмма классов